Multi-Agent AI Based Product Recommendation and Analysis in E-commerce to Improve Customer Experience

Bachelor Thesis

Submitted to the Department of Business Informatics
At the Faculty of Management Technology
German University in Cairo

Supervisor: Dr. Gamal Kassem

Presented by: Hesham Amr Mohamed El-Nabawy

ID: 58-26237

Submitted On: 14/05/2026

Table of Contents

List of Abbreviations

AI	Artificial Intelligence
API	Application Programming Interface
B2B	Business to Business
B2C	Business to Consumer
CBF	Content-Based Filtering
CF	Collaborative Filtering
CPU	Central Processing Unit
CRISP-DM	Cross Industry Standard Process for Data Mining
CSV	Comma-Separated Values
CX	Customer Experience
E-Commerce	Electronic Commerce
GNN	Graph Neural Network
GPU	Graphics Processing Unit
JSON	JavaScript Object Notation
KDD	Knowledge Discovery in Databases
LLM	Large Language Model
LoRA	Low-Rank Adaptation
MacGNN	Macro Graph Neural Network
RAG	Retrieval-Augmented Generation
RAM	Random-Access Memory
TOE	Technology-Organization-Environment
UI	User Interface

List of Figures

List of Tables

1 Introduction

With the evolution of technology and e-commerce websites, the number of products available online has vastly increased, making it harder for customers to navigate the websites efficiently, and as the digital marketplace expands, organizations are not only relying on pricing and availability but also on customer experience (CX) as CX quality has become essential to keep an edge over competitors. By integrating Artificial Intelligence (AI) into recommendation systems, it has made them more personalized and dynamic, making them an essential part of business strategies in e-commerce (Yin et al., 2025).

1.1 Motivation

This thesis is motivated by the shift toward AI-based recommendation systems in the field of e-commerce. Traditional recommender systems using collaborative filtering (CF) and content-based filtering (CBF) have worked very well. However, they have not advanced much in recent years as they hit their performance ceiling, especially with their reasoning ability. Therefore, the integration of LLMs into recommender systems offers a huge opportunity for improvement with their reasoning ability and their ability to understand the user's intent and provide reasoning for their recommendations (Zhang et al., 2025).

But the problem is that integrating these systems also brings challenges related to hallucinations, security and ethical concerns (Madanchian, 2024). Therefore, understanding the foundations of AI is critical to using these technologies efficiently. This paper is motivated by understanding these foundations to apply them correctly to be able to increase the overall customer experience in the e-commerce business.

1.2 Problem Description

The main problem with integrating AI technologies lies in balancing the benefits with the risks. Although traditional methods like CF are stable, they still suffer from limitations including dealing with cold start customers due to relying on historical data (Zhang et al., 2025). Moreover, they lack reasoning which could improve customer experience, they also suffer greatly with data sparsity (Zhang et al., 2025). While newer methods like Graph Neural Networks (GNNs) have solved some of these issues, they still struggle with scalability and efficiency due to their high computational costs (Karan et al., 2024).

LLMs solve these problems. However, they come with new ones, mainly hallucinations, which in the context of e-commerce recommender systems means that a model recommends items or products that do not exist or invents new product features, this is a severe risk as it can make customers lose their trust, which leads to worse customer experience that directly leads to losing customers (Larsen et al., 2025). Systems using

LLMs must therefore balance reasoning and personalization with the strict accuracy needed to retain customers' trust.

1.3 Objective of this Thesis

The primary objective of this thesis is to build a recommendation system that can use the reasoning and personalization capabilities of LLMs while making sure that all recommendations map to existing inventory items to mitigate hallucination.

1.4 Scientific approach

The scientific approach used in this paper is our solution approach based on the design science research derived from the methodology of Offermann et al. (2009). This solution approach is structured into three phases. The first phase is problem identification where it conducts a literature review that extracts a scientific gap from state-of-the-art papers that are grouped into different clusters and then analyzed to extract a clear scientific gap and research question. The second phase is design where partial solutions to the same gap are analyzed to extract functional and non-functional requirements and then design both a structural and behavioral model. The final phase is evaluation where the solution is implemented, demonstrated and then evaluated both quantitatively against the requirements and qualitatively using feedback from expert interviews.

1.5 Chapter Outline

This paper is organized as follows:

Chapter 2 provides theoretical definitions and history of Artificial Intelligence, including overviews of LoRA, RAG, and AI in general.

Chapter 3 focuses on e-commerce and recommender systems, exploring the evolution from traditional systems such as CF and CBF to newer ones including GNNs and LLMs.

Chapter 4 discusses where the e-commerce data comes from and what the most important aspects of that data are, and how to clean this data to ensure consistency and how to ensure privacy, it then discusses how the recommender systems and CX are scientifically evaluated to improve them.

Chapter 5 presents a detailed literature review that explores current state-of-the-art academic papers and analyzes them to extract a clear scientific gap of using AI in e-commerce recommender systems to improve customer experience and the research question.

Chapter 6 presents the most popular methodologies in design science research (Hevner et

al., 2004), (Peffers et al., 2007) and (Offermann et al., 2009) and then discusses the difference between qualitative and quantitative studies and other popular methodologies.

Chapter 7 presents our solution approach that will be followed to find the problem and design a solution to solve this problem and after that how to evaluate the proposed solution.

Chapter 8 is the results chapter where the solution approach is applied from start to finish from the problem identification to the design phase where partial solutions are studied to find the requirements for the solution and finally model the solution both structurally and behaviorally.

Chapter 9 starts with the implementation of the solution from the design phase starting with the tools used and installation steps to the demonstration of the solution and then evaluating the solution both quantitatively against the requirements and qualitatively using expert interviews and refining the solution based on their feedback.

Chapter 10 is the conclusion chapter which summarizes the paper from the objective onward and presents the limitations in the solution and future work.

2 Artificial Intelligence Overview and Role in Organizations

This chapter presents the theoretical definitions and technological advancements of Artificial Intelligence, providing an overview of terminologies and risks of adopting AI.

2.1 Overview and Definition

Despite the word intelligence being used often, it is hard to describe it in a single word, as there is no globally agreed upon definition, instead it is a list of characteristics that AI is trying to adopt, like autonomy, which is to take decisions and actions without explicit direction, or communication, being able to communicate goals, beliefs and course of action, and many others of course (Oliveira, 2019; p. 1-2).

Same as before, AI does not have a single definition, but the many definitions can be summed into this "to enable human intelligence to be equipped with machines by understanding human judgment, behavior, and cognition." (Garg, 2021; p. 1), moreover, it is the ability of machines to perform human-like tasks.

Others define AI based on its specific tasks, like the ability to learn from given feedback, or the ability to set certain goals and pursue them, but AI is still something that is difficult to give a clear definition of, as it is based on something we still do not fully understand that is human intelligence (Sheikh, Prins & Schrijvers, 2023; p. 15-16).

2.2 A brief History of AI

As a field in the academic world AI dates all the way back to the 1950s, with the term first being officially introduced in 1956 as part of a program in Dartmouth, this program explored the possibility of machines replicating humans' cognitive ability (Benbya, Davenport & Pachidi, 2020; p. 2), two years later John McCarthy introduced a programming language specifically for AI called LISP, which was used for the next 30 years after its release.

Unfortunately, no further advances were made in the years that followed, as the funding for AI research was heavily cut down due to critics, but AI started to make a comeback when Google developed AlphaGo in 2015, a program that was able to beat the world champion in the board game go, a game much more complex than chess (Haenlein & Kaplan, 2019; p. 4).

2.3 AI in Organizations

The introduction of AI had a significant effect on organizations, as it increased the need for and importance of technology experts like data scientists and machine learning experts, which can shift the authority in the workplace and give technology practitioners increased control over the decision-making process. (Benbya, Davenport & Pachidi,

2020; p. 9-10)

AI in organizations has a lot of future opportunities, as machine learning tools can improve the productivity of data scientists, and make sure inexperienced data scientists can complete their projects, but a limitation is how much data is needed to train the models. Some researchers argue that it is unsustainable. (Benbya, Davenport & Pachidi, 2020; p. 11)

2.4 LLMs Overview

Large Language Models outline a critical shift in AI, moving away from basic pre-trained language models to models that output logical text while handling various tasks (Naveed et al., 2025; p. 2). In theory, this marks progress from simple pattern recognition to something closer to human understanding, allowing models to imitate human thinking and adapt accordingly (Wu et al., 2025; p. 2).

Modern LLMs use the Transformer structure; it relies on self-attention to capture context and long-range word links (Wu et al., 2025; p. 8). Development starts with large-scale unsupervised learning across varied texts, this helps models pick up language rules, syntax, and basic world knowledge (Wu et al., 2025; p. 2). Instead of stopping there, they go through targeted tuning phases, so models can specialize for specific tasks. For specialization, two main approaches apply. First is training on annotated examples (Supervised Fine-Tuning) or improving responses via feedback signals using reinforcement methods (Reinforcement Learning from Human Feedback) (Naveed et al., 2025; p. 7). Due to full updates demanding heavy computing power, efficient alternatives exist such as LoRA that adjusts just small parts of the network. These saving strategies fall under PEFT, largely decreasing resource needs (Naveed et al., 2025; p. 20).

Figure 1 shows a basic flow diagram of the various stages of LLM, starting from pre-training to prompting/utilization (Naveed et al., 2025; p. 6).

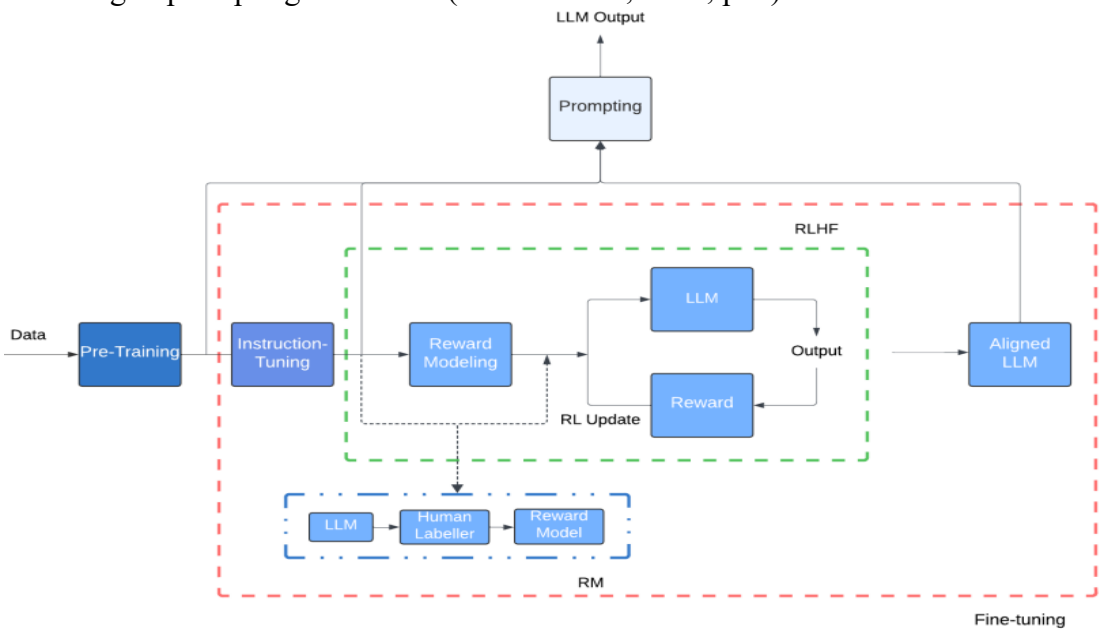

Figure 1: LLM stages (Naveed et al., 2025)

Smart prompt methods are used to help improve LLM performance. For example, Chain-of-Thought prompts help models reason in steps. Moreover Retrieval-Augmented Generation increases reliability by pulling data from external knowledge sources (Naveed et al., 2025; p. 7). Therefore, these systems can be used to support real world tasks such as helping with diagnoses, helping banks evaluating financial threats or helping with software development (Wu et al., 2025; p. 11).

2.5 RAG in Organizational Context

Retrieval-Augmented Generation (RAG) is a hybrid model designed to overcome the limitations of pure generative models, RAG integrates a retrieval mechanism from an external knowledge source and a generation module, that processes information generated to generate human-like text (Gupta, Ranjan & Singh, 2024; p.2).

A traditional RAG process works by giving an input query, then the retriever identifies relevant data sources. The retrieved information then interacts with the generator to improve the generation process as depicted in **Figure 2** (Zhao et al., 2024; p. 1-2).

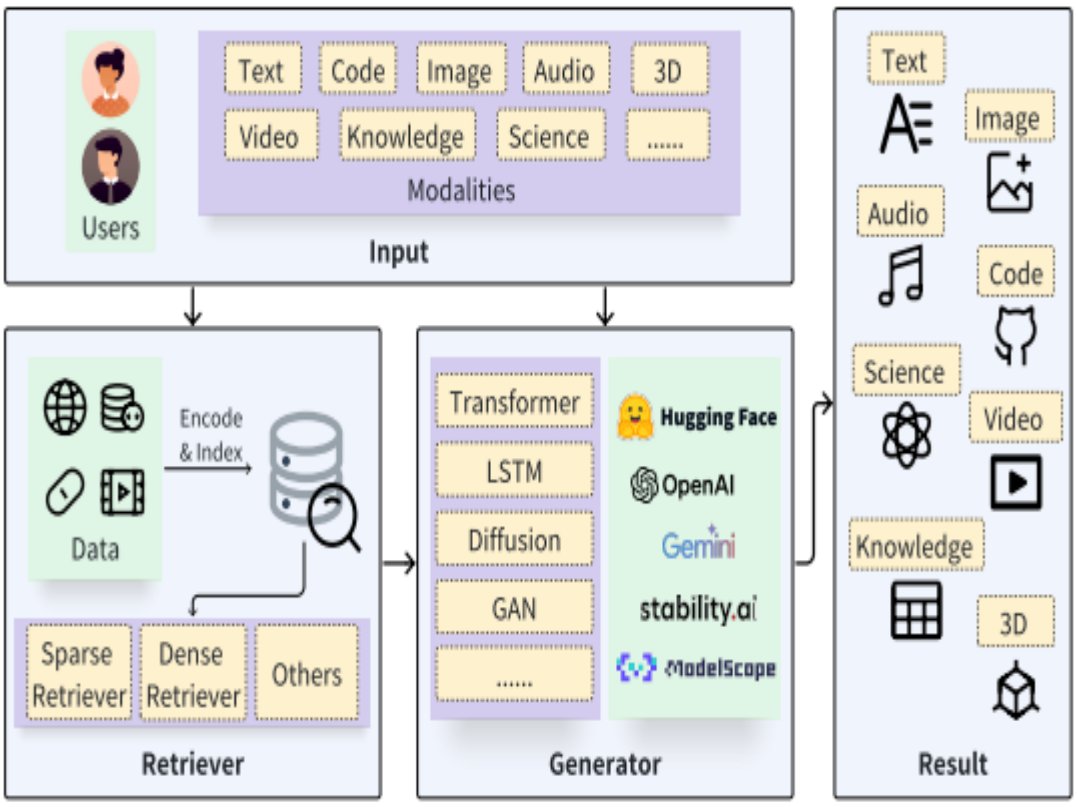

Figure 2: Typical RAG architecture (Zhao et al., 2024)

RAG models have many applications; it can be used in medical diagnosis systems to retrieve the latest research and use it for accurate diagnosis for patients. Moreover, it can be used in personalized recommendation systems to retrieve and use preferences to generate personalized recommendations (Gupta, Ranjan & Singh, 2024; p. 4).

2.6 LoRA in Organizational context

Low-Rank Adaptation (LoRA) is an approach to model adaptation that enables training dense layers in a neural network by optimizing rank decomposition matrices of dense layers during adaptation, while keeping the pretrained weights the same, as shown in **Figure 3** (Hu et al., 2021; p. 2).

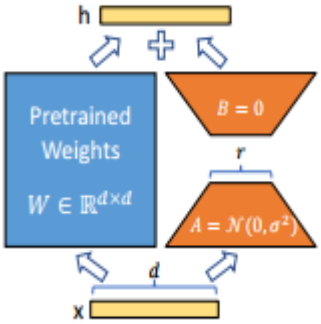

Figure 3: Reparameterization. Only matrices A and B are trained. (Hu et al., 2021)

LoRA has multiple advantages such as lowering the hardware requirements needed when using adaptive optimizers up to 3 times and making the training more efficient overall. Moreover, pretrained models can be used to build many small LoRA modules for various tasks (Hu et al., 2021; p. 2).

All these benefits of using LoRA make it very enticing for organizations as it can increase the overall efficiency while reducing cost.

2.7 Risks and Ethics of AI adoption

AI has many ethical concerns behind it, some are caused by the features of AI and some are caused by human factors and many others, Data security and privacy are huge concerns as the performance of an AI model largely depends on the training data, which can include personal and private data and with that comes the risk of information leakage or tampering which can be a huge ethical issue. In addition, responsibility can be an issue as an AI is given responsibility to perform certain tasks which can be an issue if given too much of it. Moreover, ethical issues can also be caused by human factors, with one of the main issues being accountability, as if an AI fails in a task assigned by a human and that leads to bad consequences, who should be responsible. (Huang et al., 2023; p. 802)

A major risk of adopting AI is interaction issues, as human and machine interact challenges appear such as automated manufacturing, transportation and infrastructure systems. If the operator of the automated machine or vehicle is not paying attention that could cause major accidents and injuries. (Cheatham et al., 2019; p. 4)

Szadeczky & Bederna, (2025) classify AI risks as minimal, limited, high and unacceptable risks, with high-risk applications including AI applications that could affect the health and safety of people or groups negatively.

2.8 Summary

This chapter aims to give a general introduction to the primary principles of Artificial Intelligence. The complexity of defining intelligence and the evolution of AI is discussed, ranging from its creation in the 1950s to the present day of Generative AI. Moreover, LLMs are explored, defining them and their use cases, and how they are trained and fine-tuned. The role of AI within organizations, the impact on workforce roles and the

technical application of models is also discussed, Retrieval-Augmented Generation (RAG) and Low-Rank Adaptation (LoRA) are highlighted as methods to enhance efficiency. Additionally, the ethical implications and risks associated with AI adoption, ranging from data privacy to accountability and safety, are considered.

This introduction is necessary since advanced organizational implementations depend basically on the concepts and technical capabilities of these models. Therefore, it is essential to have a solid background about these technologies, their benefits, and their risks before going deep toward the details of practical application. Which is the subject of the next chapters, discussing how AI can be used to enhance the recommender systems to improve the overall customer experience when using e-commerce websites.

3 E-commerce and Recommender Systems

After chapter two introduced AI and outlined its basic concepts, this chapter is focused on e-commerce, recommender systems and how AI integrates into them.

3.1 E-commerce Overview

E-commerce is electronic commerce, which is the media and internet way for the dealing of goods and services, and it redefines value generation relations between organizations and individuals. It concerns a vendor website on the internet that trades goods or services to the customers directly from the website, goods are often put into purchase baskets then paid for by credit and debit cards or other forms of electronic fund transfer. (Jain, Malviya & Arya, 2021; p. 665)

E-commerce has six types as shown in **Figure 4**, with the one that concerns this paper the most being Business-to-Consumer (B2C), it is the shopping section of e-commerce where traditional retail businesses mostly take place. (Jain, Malviya & Arya, 2021; p. 666)

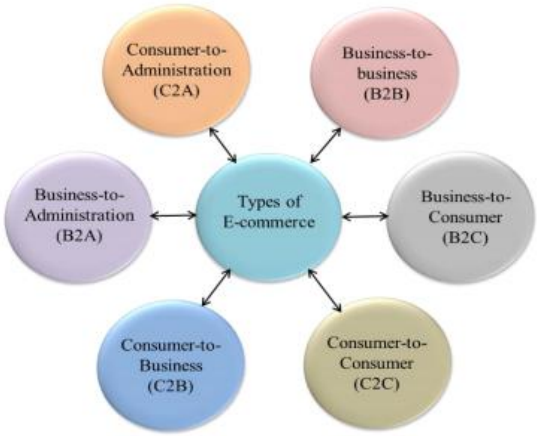

Figure 4: Schematic illustration of the Types of E-commerce (Jain, Malviya & Arya, 2021)

3.2 Recommender Systems Overview

Recommender Systems have become a big research interest that focuses on helping users

find products or services online that closely align with their interests, it does that by a smart computer-based method that predicts consumers' usage to help them pick from the huge selection of products. This has helped both the customers and the businesses in e-commerce by increasing sales while giving the customer better experience. In addition, recommender systems are also used in other knowledge-based applications such as online news portals and social media (Singh et al., 2021; p. 15-16).

There are many types of recommender systems as technology advances such as content-based filtering (CBF), that collects features of an item that the user purchases or gives a positive rating to and recommends items with similar attributes. For example, if a user likes a movie, movies with same directors, actors and or genre will be recommended to that user. Furthermore, there's collaborative filtering (CF) which works by measuring the similarity between users, by making a collection of users that have similar preferences and dislikes, and recommending the items preferred by most users in the group to other users in the same group. (Roy & Dutta, 2022; p. 3-4)

Despite the success of recommender systems, they still face many challenges that must be dealt with to improve their performance. Including data bias, as new recommender systems acquire user data, this can lead to bias from the acquired data, which reduces the model's performance and affects the user experience (Li et al., 2024; p. 11). In the next chapters, recommender systems are discussed in more detail based on their technologies.

3.3 CF, CBF and Hybrid Recommender Systems

Collaborative Filtering (CF) can be divided into two categories, model-based and memory-based methods. Memory based methods are more documented due to their efficiency and effectiveness, it uses the similarities between users or items to make recommendations. CF recommends items based on a simple algorithm that can give accurate results; however, it is limited due to the high sparsity of data. (Aljunid & Dh, 2020; p. 830). However, CF faces multiple challenges including dealing with cold start customers, where there's limited data available for new users or items, moreover, it faces popularity bias, where popular items receive more recommendations. (Widayanti et al., 2023; p. 291)

In content-based filtering (CBF), recommendations given to users are dependent on the users' past choices. Users' profiles and item descriptions of past choices are critical in CBF, it also uses similarity count of items. (Thorat, Goudar & Barve, 2015; p. 1) CBF also faces challenges, with mainly being the lack of diversity as it only focuses on recommending items with similar characteristics and it tends to ignore user preferences, however it deals with the cold start problem better as it does not rely on historical user data (Widayanti et al., 2023; p. 292). Current recommender systems tend to use hybrid approaches, which combines CF, CBF and other approaches. One popular example is Netflix which provides recommendations by offering movies with similar attributes

(CBF) and comparing watching habits of users with similar interests (CF). This approach combines both the advantages of CF and CBF and other approaches to provide better recommendations and overcome limitations. (Widayanti et al., 2023; p. 296-297)

3.4 Graph Neural Networks

Graph Neural Networks (GNNs) are a deep learning framework made to work on graph structured data; it offers an advantage over traditional methods by capturing both spatial relationships and temporal patterns at the same time. GNNs model dependencies and interactions between nodes, this allows GNNs to integrate complex multi-channel relationships, making GNNs a great tool for improving accuracy. (Wang et al., 2024; p. 1-2) making them ideal for recommender systems.

GNN based recommenders have become very successful for recommender systems and it can be explained from three aspects.

Firstly, structural data, as data collected from e-commerce platforms come in many forms such as user-item interactions, user profiles and item attributes, with traditional recommender systems not being able to use those multiple forms of data, making them focus on few specific sources which hinder performance. GNNs solve that by being able to express all the data as nodes and edges on a graph, giving higher recommendation performance. (Gao et al., 2023; p. 16)

Secondly, high-order connectivity, accurate recommendation systems rely on effective embedding spaces that capture both direct user-item interactions and the "collaborative filtering effect," where items preferred by users with similar interests are relevant. While traditional models are often limited to "first-order connectivity" or direct interactions, GNNs improve on that by modeling "high-order connectivity." By propagating embeddings across multi-hop neighbors in the graph, GNNs naturally incorporate these complex, indirect relationships, therefore overcoming the limitations of traditional approaches and improving recommendation performance relative to traditional methods. (Gao et al., 2023; p. 16)

Finally, supervision signals, GNN based recommendation systems can use multiple signals for recommendations, as in e-commerce the main target behavior is purchase, but there are many other behaviors such as search and add to cart. By using multiple signals other than the target behavior, they achieve higher recommendation performance. (Gao et al., 2023; p. 16)

Applying GNNs to recommender systems has multiple challenges including computation efficiency as compared to traditional methods computational cost of training GNNs is a lot higher. Moreover, graph construction can be challenging to do it to handle the task correctly and needs to be carefully implemented with consideration to nodes and edges. (Gao et al., 2023; p. 17)

3.5 LLM Recommender Systems

Large Language Models (LLMs) revolutionized the technological field with their outstanding generalization abilities and reasoning skills, and recently they have become critical in enhancing recommendation systems with their ability to shorten the gap between research and deployment, enabling real translation of recommender algorithms into real-world applications. (Wang et al., 2024; p. 1-2)

LLMs have many advantages that could benefit recommender systems including the ability to deal with cold start users, due to their large world knowledge and better ability to understand product descriptions, LLMs could decrease the cold start problems. Moreover, recent advancements in LLMs show high potential for customization, as each platform requires unique requirements such as e-commerce, which need platforms that handle changes in pricing and seasonal trends and a wide variety of products. (Wang et al., 2024; p. 27-28)

However, like any new technology LLMs face their challenges such as efficiency, as to correctly implement LLMs in recommender systems involves managing computational costs and optimizing performance which can be a challenge. In addition, fairness can be an issue as they need to mitigate bias to different age, gender and race groups. (Wang et al., 2024; p. 30-31)

In addition, hallucination is a problem with LLM-based recommender systems, it refers to a model generating items that are not grounded in the input or real-world, this can degrade the accuracy of LLM-based recommender systems, when the system faces hallucination it can produce identical recommendations when given contradictory prompts, which severely undermines the reliability of these systems. (Deng et al., 2025; p. 29301)

3.6 Summary

This chapter introduces the concept of e-commerce and the critical role of recommendation systems. The definition of e-commerce was discussed with a focus on the B2C models. Moreover, this chapter discusses the evolution of recommender systems from traditional methods such as Collaborative filtering and Content-Based filtering to hybrid systems that combine both to minimize their limitations and capitalize on their strengths. Furthermore, more advanced frameworks were explored such as Graph Neural Networks that plot data on a graph and connect them to predict and recommend items. Then, Large Language Models, that have advanced reasoning and generalization skills but face problems like hallucination and computing costs.

After understanding the different types of recommender systems and their strengths and limitations, the next chapter explores where the data used by these systems is retrieved from, and how these recommender systems are evaluated to further improve on them. In addition, it discusses how customer experience is evaluated, to further help with the improvement of these systems using the feedback from CX.

4 Data Sources and Evaluation

This chapter explores the data sources that the recommender systems use and how to make sure that this data is reliable and consistent. Then, how are the recommender systems evaluated and how the customer experience is evaluated.

4.1 E-Commerce Data Sources and Schemas

E-commerce platforms deal with two types of data, structured and unstructured. Structured data includes demographic data such as name, age, address and preferences. While unstructured data focuses on clicks, links, posts etc. (Akter & Wamba, 2016; p. 176)

In context of recommendation systems, they find patterns and offer personalized recommendations by using user data which includes user searches, purchases, browsing behaviors, ratings and wish-listed products. (Ejjami, 2024; p. 2)

Kohavi et al. (2004) Discusses that the data collection architectures collect unique "Business Events" that include searches and number of results returned, that allows them to identify failed searches that return zero results. Moreover, shopping cart events such as adding to cart, change of quantity and removing from cart that are essential for tracking shopping cart abandonment. In addition, checking out and order confirmation are collected too.

4.2 Data consistency

Before the data is used for training the recommendation systems, it must be ensured that it is consistent and accurate, to prevent further errors.

Robust data integration and error handling mechanisms are essential. Moreover, there's a need for enhanced security measures and transparency in the data governance practice to prevent security and privacy issues as this is sensitive customer information. (Reddy & Nalla, 2024; p. 301)

Data cleaning is essential before using it to ensure data consistency and high-quality outputs from recommendation systems.

Data cleaning is composed of multiple stages, starting off with identifying and handling null values as they can skew analysis. This is done by filling in missing values based on

existing data (imputing). After that, inconsistencies should be detected and corrected, these include incorrect formatting, typos and variations in units of measurements. Finally, duplicates are removed as they inflate results, this ensures clean and consistent data that can safely be analyzed and modeled and further used for training recommender systems. (Sharma et al., 2025; p. 5-6)

4.3 Recommender System Evaluation

Once the data is gathered and used to train the recommender systems, multiple evaluation methodologies are used to assess the effectiveness of the recommender system.

One of them is the rating-based indicator, that focuses on the accuracy of the predicted values, the system tries to predict the rating the user would give to the predicted item. With the goal being to minimize the gap between the actual rating provided by the user and the predicted rating by a mathematical equation. (Li et al., 2023; p. 14)

In addition, there is the item based indicator, that differs from the rating-based indicator by assessing the quality of the recommendation list as a whole, the system is evaluated on its ability to recommend a relevant list of items that the user is likely to interact with and is more aligned with real-world e-commerce scenarios, with a primary objective of presenting a top-n list of products that leads to click or a purchase. (Li et al., 2023; p. 14)

To be able to evaluate the methodologies discussed above, performance metrics based on mathematical basis are used. For rating-based evaluation error metrics are used, particularly Root Mean Squared error, which calculates the square root of the average differences between the true ratings and predicted ones, however it is very sensitive to outliers. (Li et al., 2023; p. 14)

As the order of recommendations is very important, normalized discounted cumulative gain is used, with the goal of putting more relevant items at the top of the list. However, having incomplete ratings cannot be handled by it. (Li et al., 2023; p. 14-15)

Khan & Sajjad (2024) finds that data privacy and security concerns could deter customers from sharing their data. Therefore, the paper proposes that using privacy-preserving technologies like differential privacy and federated learning offer exciting solutions for e-commerce businesses.

4.4 Customer Experience Evaluation

Customer Experience (CX) is a primary concern of e-commerce, and it represents an important driver of customer satisfaction, business goals and loyalty. CX evolved from just post-purchase satisfaction into a multidimensional construct formed by digital touchpoints, personalization, emotional engagement and adaptive technologies.

(Mittameedi, Dogra, & Sayal, 2025; p. 154422)

Huang & Wang (2022) evaluate the CX by using questionnaires, by first collecting demographic data about the users, then using interviews and questionnaires to understand the users' feelings. It uses questions related to overall satisfaction and possible user worries. Including data about users' attitudes, usage, problems and expectations with products.

Mittameedi, Dogra, & Sayal, (2025) explores multiple methodologies of evaluating CX in e-commerce. Including sentiment analysis, which captures the emotional tone of the customers by analyzing feedback from digital channels. However, as it relies on text analysis it can miss details.

Another method is customer journey mapping, which identifies key touchpoints by visualizing the customers' end-to-end interaction, but it can be time consuming as it requires using detailed data. (Mittameedi, Dogra, & Sayal, 2025; p. 154429)

There's also the customer experience index, which provides a single score as a composite metric for overall CX. However, it is not as detailed as other methods and may not capture all areas. (Mittameedi, Dogra, & Sayal, 2025; p. 154429)

4.5 Summary

This chapter explores how the data used in recommender systems for e-commerce is gathered. Furthermore, it distinguishes between structured and unstructured data, and the importance of certain events like adding to cart. Moreover, how this data is processed and cleaned before being used in those systems to prevent any inconsistencies, and how to make sure that the customers trust submitting their data by using privacy-preserving technologies. After that, this chapter discusses how the recommender systems are evaluated to ensure proper performance by using rating-based and item-based indicators. Finally, the evaluation of CX was discussed through multiple methods like sentiment analysis and the use of questionnaires to assess user data.

The next chapter presents a structured literature review to discuss the current academic research regarding AI-based recommender systems in e-commerce, by dividing the chapter into 3 sections discussing the state-of-the-art papers, analyzing them and finally it finds the scientific gap of this research.

5 Literature Review

In this Chapter, a structured literature review is conducted to examine recent and relevant academic research to identify a scientific gap for using AI based customer recommendation systems in e-commerce, this review is divided into two stages. First, state of the art papers are collected according to a defined methodology and divided into three clusters based on what method they use to address the thesis topic.

A total of eight different papers are discussed, that tackle different methods to enhance AI in recommender systems.

Second, in the discussion section the clusters are compared with their findings and limitations to find a scientific gap behind using AI based product recommendation in e-commerce to improve customer experience.

5.1 State of The Art

In this chapter state of the art academic papers are chosen according to the selection methodology which is:

- **Recency:** All selected literature was published between 2023-2025.
- **Database searched:** All articles chosen are obtained from Google scholar and arXiv to maintain academic and citable articles.
- **Keywords used:** For each cluster different keywords were used corresponding to them:
    - **Cluster 1 (Traditional collaborative filtering approach for recommendation systems in e-commerce):**

      Keywords: “collaborative filtering” AND “e-commerce”
      Excluding: “AI”, “Deep Learning”
      Filter: Must appear in the title
      Result: 70 papers
    - **Cluster 2 (Graph Neural Network Approaches in Recommendation Systems):**

      Keywords: “Graph neural networks” AND “recommender systems”
      Filter: Must appear in title

Result: 36 papers

Cluster 3 (Intersection of AI recommender systems in e-commerce to improve customer experience using LLM):

Keywords: “LLM” AND “recommender”

Filter: Must appear in title

Result: 112 papers

- **Post selection**: After getting the results, articles were post filtered by removing duplicates and trivial papers, then selecting papers depending on their relevancy to the topic of AI in e-commerce recommender systems yielding a total of eight papers.

5.1.1 Cluster 1: Traditional collaborative filtering approach for recommendation systems in e-commerce

This cluster groups together research that explore the use of traditional methods such as collaborative filtering and content-based filtering for recommendation systems in e-commerce, which were selected by using the keywords “collaborative filtering” and “e-commerce”.

Recommendation System for E-Commerce Using Collaborative Filtering (Patil et al., 2024)

The study "Recommendation System for E-Commerce Using Collaborative Filtering" by Patil et al. (2024), explores key issues in traditional collaborative filtering, such as data sparsity and cold-start user challenges, by combining CF with CBF. Instead of only relying on one technique, the team built a hybrid model using both Collaborative Filtering (CF) and Content-Based Filtering (CBF). They worked with the "Marketing Sample for Walmart.com" set, made up of 5,000 entries showing how users interacted with items. Before analysis, they cleaned the data using strong scaling procedures along with Singular Value Decomposition (SVD) to cut down variables and handle outliers. Their main method applied Item-Based CF, where Truncated SVD broke down user-product rating patterns, keeping only links stronger than 0.90. At the same time, CBF used TF-IDF scoring plus Nearest Neighbors logic to recommend goods matching past user choices. Results showed this combined setup reached 97% accuracy, delivering solid precision and recall scores while beating other approaches including Linear SVMs and Decision Trees. Although the system improved personalized recommendations and mitigated cold-start challenges, it had its limitations, like possible data selection flaws and no support for live context such as location or gadget used. Looking ahead, researchers argue that combining neural networks, text analysis tools, besides ongoing user input might boost how well the platform adjusts within fast-changing online shopping environments.

Product collaborative filtering based recommendation systems for large-scale E-commerce (Trinh et al., 2025)

The study “Product collaborative filtering based recommendation systems for large-scale E-commerce” by Trinh et al. (2025) explores performance issues in e-commerce recommender systems, especially when handling large data sets while delivering fast and personalized recommendations. Instead of standard setups, the team introduces a new method combining Alternating Least Squares (ALS) matrix decomposition with distributed processing through Apache Spark on Google Cloud, to test their new method, they used a collection of Amazon user reviews, (specifically the 5-core subset including categories like "Arts, Crafts and Sewing," "Pet Supplies," and "Tools and Home Improvement"), to both train and evaluate their system. Their method, using ALS with Spark across multiple nodes, was measured against traditional models by using K-Nearest Neighbors (KNN) algorithms. The findings were significant, showing the parallel setup sped up training nearly 7.6x versus standalone systems yet kept strong performance with a Root Mean Square Error (RMSE) of 0.9. Importantly, the clustered framework handles large inputs like "Pet Supplies" and "Tools", where single units failed or timed out, highlighting better scaling capability. A key contribution here is confirming distributed computation works well for e-commerce data challenges, giving quicker, cheaper results than standard techniques while preserving forecast precision. However, limitations in the available equipment limit trials on broader datasets. Moreover, though RMSE values were good, they weren’t fine-tuned for the best commercial value. Future researcher recommendations suggested by (Trinh et al., 2025) include testing on larger datasets, integrating multi-modal approaches to enhance predictive accuracy and optimizing configuration and hyper parameter tuning to align computational efficiency with costs.

Cluster Summary

The papers discussed in this cluster by (Trinh et al., 2025) and (Patil et al., 2024) discuss the use of collaborative filtering for recommendation systems in the field of e-commerce to improve user satisfaction, both papers used different methodologies to deal with the limitations of CF-like scalability. (Trinh et al., 2025) developed a scalable framework based on CF using a matrix factorization method in Apache spark to improve accuracy and speedup processes. On the other hand, (Patil et al., 2024) used a hybrid system merging CF with content-based filtering (CBF) to achieve a high accuracy rate of 97% but is tested on a small dataset of only 5000 rows risking sampling bias. This cluster outlines the weaknesses of CF with cold start customers, while the hybrid system attempts to solve it, it still is a large limitation of traditional CF, as they heavily rely on historical data like user-item interaction. Moreover, CF struggles with data sparsity which requires sophisticated techniques like matrix factorization to solve it and struggles with accuracy with shifting user preferences (Temporal dynamics) outlined in the hybrid model by (Patil et al., 2024).

5.1.2 Cluster 2: Graph Neural Network Approaches in Recommendation Systems

This cluster groups together research that explore using Graph neural networks in recommendation systems, which were selected using the keywords “Graph neural networks” and “recommender systems”

A Survey of Graph Neural Networks for Recommender Systems: Challenges, Methods, and Directions (Gao et al., 2023)

This survey by Gao et al. (2023) discusses how Graph Neural Networks (GNNs) can improve recommender systems. GNNs can do that by focusing on overcoming the weaknesses of traditional shallow and neural models by modeling high-order connectivity and structural data features. The survey categorizes previous work into four different views, including phases such as matching, ranking, or re-ranking, another viewpoint covers use cases such as social recommendations, sequence-driven setups, session contexts, bundled items, transfers across domains, or users showing multiple behavior types. Objectives of this survey include accuracy, diversity, transparency, and fairness among user groups. This framework reveals that GNNs improve system capabilities by modeling collaborative filtering effects via multi-hop neighborhood propagation, enabling nodes to access high-order information missed by traditional methods. For instance, during matching stage, GNNs use embedding propagation to identify candidates. In contrast, in the ranking phase, models apply encoder-predictor designs to detect complex relationships. The main advantage of GNN-based methods is their ability to use multi-modal structural data while improving training signals which helps reduce problems such as data sparsity. However, the survey points out limitations, including the challenge of building accurate graphs from unstructured information. Over-smoothing is a problem that restricts the depth of GNN layers and embedding propagation has high computational cost, making it hard to scale for live industry use. Moreover, the use of graph structures can increase fairness concerns by propagating algorithmic bias. In conclusion, researchers future research directions of developing deeper models for complex patterns, designing adaptive GNNs for time-varying datasets, along with integrating automated machine learning (AutoML) techniques and self-supervised training to refine model architecture and improve reliability.

Beyond-accuracy: a review on diversity, serendipity, and fairness in recommender systems based on graph neural (Duricic et al., 2023)

This review conducted by (Duricic et al., 2023), was done with the objective of evaluating Graph Neural Networks (GNNs) based on “beyond-accuracy” dimensions that directly impact the user experience and ethical system deployment. Specifically, the authors aim to provide an overview of how diversity, serendipity and fairness are dealt with in GNN frameworks. The methodology was starting a manual literature search and selection

process, reviewing 20 publications from relevant journals and conferences that address these metrics in GNN recommender systems, they did a multi-stage model development process that includes data preprocessing, graph construction, embedding initialization, propagation layers, embedding fusion, score computation and training methodologies to locate where interventions are made to enhance beyond-accuracy metrics. The results of this paper show that GNNs have high accuracy in collaborative filtering. Moreover, for diversity strategies include neighbor-based mechanisms, dynamic graph construction to capture non-interactions and adversarial learning to help reduce bias. In serendipity, few studies were found that explore it but outlined that enhancements come from neighbor-based mechanisms that give the users the ability to control explore-exploit trade-offs, that boost serendipity. The review highlights efforts to control user and item bias through debiased neighbor aggregation which helps with fairness. However, this review acknowledges limitations such as the "accuracy-diversity tradeoff" where optimizing for fairness or diversity can decrease accuracy and lead to overfitting due to the complexity of GNN structures. Future research recommends improving personalized recommendations instead of global preferences and more robust models to handle data sparsity.

Macro Graph Neural Networks for Online Billion-Scale Recommender Systems (Chen et al., 2024)

This paper by Chen et al. (2024) titled "Macro Graph Neural Networks for Online Billion-Scale Recommender Systems" explores the high computational costs and bias in integrating Graph Neural Networks (GNNs) to billion-scale recommender systems. The main objective of this paper was to overcome the weaknesses of existing recommendation graphs, which needs to sample a tiny fraction of neighbors from billions of interactions. This paper aims to solve this issue by introducing Macro Recommendation Graph that aggregates similar users and items into macro nodes to reduce neighbor counts from billions to hundreds without losing any patterns. To do this, the authors developed the Macro Graph Neural Network (MacGNN) that is shown in **Figure 5.**

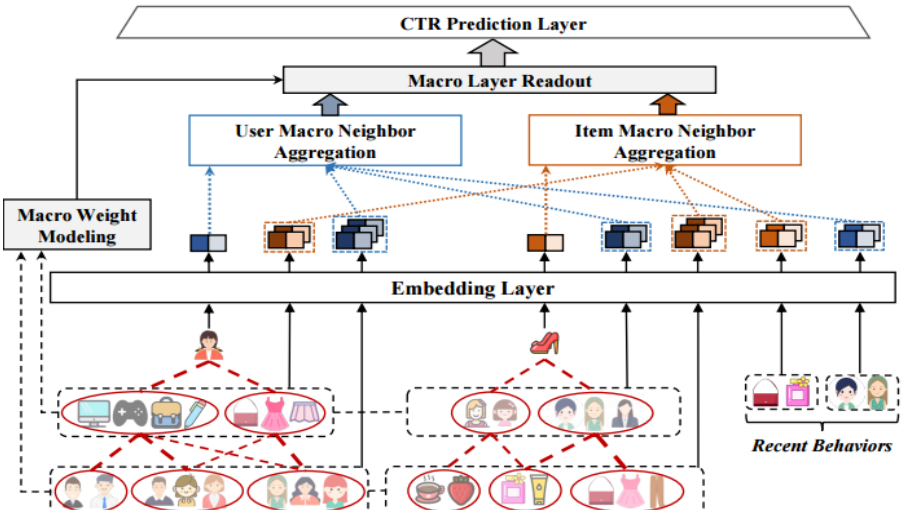

Figure 5: The architecture of MacGNN (Chen et al., 2024)

The MacGNN framework groups micro nodes based on behavioral similarity using a k-means-like cluster approach then constructs macro edges to represent aggregated interaction strengths. Moreover, this framework includes “recent behavioral modeling” component to also get the short-term interests along stable micro patterns. After that, this framework was evaluated using offline experiments on three public datasets named MovieLens, Electronics and Kuaishou and another industrial datasets. Then, A/B testing was done on Taobao’s homepage. The results of these tests showed that MacGNN outperformed 12 other baselines, achieving an average improvement of 1.00% in AUC and 1.7% in Logloss. In the online A/B tests, they served over a billion users, MacGNN increased user stay time 1.01% and gross merchandise value by 5.13% compared to the best performing baseline and reduced response time by 20%. Moreover, it eliminates sampling bias by passing every action through macro aggregation. But this method still has its limitations, including determining the optimal grouping stages and subgraphs for macro nodes as the greatly diverging node distribution between items and users complicate the sampling process. Future work was not directly discussed but implied that this macro concept could be applied to other GNN tasks.

Cluster Summary

The three papers grouped in this cluster by (Gao et al., 2023), (Duricic et al., 2023) and (Chen et al., 2024), discuss the use of Graph neural networks (GNN) in recommendation systems. (Gao et al., 2023) provides an overview that GNN takes the e-commerce data as graphs and analyzes their relationships to improve recommender systems. As traditional recommendation, it struggles to analyze the relationship between users and attributes, while GNN can do it easier. This survey organizes existing GNN based recommendation models, then compares their graph construction strategies and optimization, and outlines hopeful research directions like dynamic graph construction and more efficient architecture. Positioning GNN as strong and evolving models in recommendation systems but has limitations in scaling and other challenges including high computational cost and over smoothing. Moreover, (Duricic et al., 2023) conducts a review of GNN based recommender systems based on based on “beyond-accuracy” dimensions, which are fairness, serendipity and diversity by reviewing 20 papers, showing that GNNs increase accuracy and help with efficiency and decrease bias. While (Chen et al., 2024) shows a solution for GNNs large scalability limitations by introducing Macro recommendation graph (MacGNN) which reduces the size of the graph by clustering similar items and users into macro-nodes, which reduces the size of the graph while still keeping behavioral patterns outperforming 12 baseline models on multiple datasets and being successfully deployed on e-commerce platforms, but it shows its weaknesses with macro-node clustering as it can oversimplify complex user behavior. Together these three papers demonstrate how GNNs can be useful but still face major challenges including efficiency, personalization, overfitting and decreased accuracy when trying to improve fairness.

5.1.3 Cluster 3: Intersection of AI recommender systems in e-commerce to improve customer experience using LLM

The final cluster groups together research that employs the use of Large Language Models in recommender systems to help improve them and customer experience, which were selected using the keywords “LLM” and “recommender”.

Recommender Systems in the Era of Large Language Models (LLMs) (Zhao et al., 2024)

Recommender Systems in the Era of Large Language Models (LLMs) by Zhao et al. (2024) is a survey about the use of LLMs in recommender systems. The main objective of this paper is to analyze and categorize the various methods that use the reasoning and language abilities of LLMs such as GPT-4 to overcome the limitations of traditional recommender systems, which struggle with textual understanding, generalization of unseen tasks and reasoning. This paper uses a structured taxonomy to categorize these methods, with two main categories, ID-based methods that use LLMs to encode discrete user/item IDs and textual information enhanced methods that use LLMs to process complex textual data like reviews and descriptions. Moreover, this survey outlines three core paradigms for integrating LLMs into recommendation systems, being pre-training, fine-tuning and prompting. The results of this survey highlight that LLMs enhance recommendation systems’ performance, in explainability, conversational interfaces and zero-shot and few-shot scenarios. With key strengths being superior generalization abilities and their capacity to unify diverse recommendation tasks. But important limitations include the high computational costs and the “hallucination” problem where models generate related but unavailable limitations and the potential of bias. Future research directions suggested focus on mitigating hallucinations, ensuring trustworthiness (privacy, fairness) and enhancing efficiency of fine-tuning process.

TokenRec: Learning to Tokenize ID for LLM-based Generative Recommendations (Qu et al., 2025)

TokenRec: Learning to Tokenize ID for LLM-based Generative Recommendations by (Qu et al., 2025) explores the use of LLMs in recommender systems by specifically tackling the weaknesses of existing user and item tokenization strategies that fail to capture high-order collaborative knowledge or suffer from vocabulary explosion. The primary objective of this paper is to develop a framework that integrates collaborative filtering signals into LLMs all while enabling generalizable and efficient recommendations for unseen users and items. To do this the authors introduce a framework named TokenRec, which consists of two components, a masked vector-quantized tokenizer (MQ) and a generative retrieval paradigm. The framework leverages GNNs, specifically LightGCN, to learn high order collaborative representations of users and items. These are then processed by the MQ-Tokenizer that quantizes continuous

embeddings into LLM-compatible tokens by a k-way encoder, codebook mechanism and random masking. Then, TokenRec uses a generative retrieval approach where LLM acts like a query encoder to generate a user preference vector, that is then used to retrieve the top items from the vector database by similarity matching efficiently. TokenRec was then tested on four datasets (Amazon-beauty, Amazon-Clothing, LastFM and MovieLens-1M) and showed better performance than baselines, including traditional CF methods and other LLM-based models, with better generalization capabilities and high accuracy even for unseen users, it also had a speed improvement of 1259.81% compared to auto-regressive LLM baselines due to retrieval based output. However, this model relies on an external GNN model to provide initial embeddings, this creates a dependency on the quality of that pre-training and the complexity of tuning the hyper-parameters for the codebooks.

CheatAgent: Attacking LLM-Empowered Recommender Systems via LLM Agent (Ning et al., 2024)

This paper "CheatAgent: Attacking LLM-Empowered Recommender Systems via LLM Agent" by Ning et al. (2024), investigates the vulnerabilities of LLM recommender systems (RecSys) under black-box settings. The objective of this paper is to show that these advanced systems are vulnerable to attacks where hidden perturbations in the users' prompts can influence recommendations leading to bad user experiences. To demonstrate this, the authors propose a framework called CheatAgent that uses a LLM as an autonomous attack agent. It works in a two-step process, firstly, an insertion positioning module that identifies critical tokens in the input prompt that when perturbed, yield maximum impact on the system's output with minimal modification. Secondly, a "LLM Agent-Empowered Perturbation Generation" module that crafts these perturbations. This process is then refined through a "Self-Reflection Policy Optimization" strategy that uses prompt tuning to adjust the attacks based on feedback from the user's system. Deep experiments were conducted on three datasets (MovieLens-1M, Taobao and LastFM) by targeting two LLM recommender systems, P5 and TALLRec. The results showed the vulnerabilities of the systems, especially in their trustworthiness and robustness and shows that CheatAgent outperformed other attack methods. Limitations of this paper include the assumption that attackers can modify interaction histories and user prompts and future research directions suggest developing better defense mechanisms to secure LLM recommender systems against prompt-based adversarial attacks and exploring the vulnerability of these systems.

Cluster Summary

The three papers grouped in this cluster by (Zhao et al., 2024), (Qu et al., 2025) and (Ning et al., 2024) focus on LLM recommendation systems and the impact they can have on e-commerce. Starting with (Zhao et al., 2024) that surveys LLM recommender systems by grouping them into two different categories with one focusing on ID-based methods and another on textual information enhanced methods, showing that LLM systems outperform

traditional ones in reasoning and conversational interfaces. While (Qu et al., 2025) introduces a framework called TokenRec that bridges the gap between recommender systems and large language models, it does that by tokenizing users and items IDs in a way that large language models can understand and use, by using a masked vector-quantized tokenizer, TokenRec also includes a generative retrieval paradigm that generates a user's preference vector using LLM, then retrieves the top similar item vectors, it was then experimented on real-world datasets with results showing that TokenRec outperforms traditional and existing LLM recommender with better accuracy. Moreover, (Ning et al., 2024) investigates vulnerabilities in LLM agents by creating an attack framework called CheatAgent that uses its own LLM agent to generate subtle "adversarial perturbations" prompt-injection attacks that make the system recommend "irrelevant items that are of no interest to users", which outlines that the new systems are not robust and are vulnerable to these attacks. This cluster addresses LLM limitations as (Zhao et al., 2024) outlines the high computational costs of fine-tuning large models and the hallucination problems, where systems recommend relevant yet unavailable products. Moreover, (Ning et al., 2024) highlights vulnerabilities. But it shows its weaknesses as CheatAgent does not provide a defense mechanism against those vulnerabilities exposed and TokenRec is still dependent on external information, requiring separate update processes for generalizing new users/items.

5.2 Discussion

This section analyzes the previous three clusters and compares them, then puts the comparison in (**Table 1**) based on their used methods, strengths, limitations and how their method is affected by hallucination, allowing for the identification of the scientific gap.

Methods Applied: The system used differs clearly across the three clusters. Cluster 1 uses traditional Collaborative Filtering together with Content-Based Filtering, applying tools such as ALS matrix decomposition within Apache Spark or hybrid models blending both filtering types. On the other hand, Cluster 2 shifts focus to Graph Neural Networks, treating e-commerce information as graphs, one example is MacGNN, which simplifies graphs by grouping alike users and products into larger nodes. Meanwhile, Cluster 3 uses large language models, adopting strategies including TokenRec, a system encoding user and item identifiers via masked vector quantizers and CheatAgent, an architecture leveraging language model agents to craft input manipulations.

Strengths: In terms of performance, Cluster 1 shows strong efficiency and scalability for large datasets, its hybrid approach reaches 97% accuracy, also solving cold start issues, whereas the Spark framework helps reduce data sparsity. Moving to Cluster 2, GNN-based systems handle user-attribute connections better compared to classic techniques and reduce bias, notably, MacGNN beats 12 standard models and works well in real-world apps. As for Cluster 3, it uses LLMs to boost recommendation quality along with user satisfaction and giving advanced reasoning to the recommendations and better personalization, TokenRec is fast, effective, and surpasses traditional as well as current LLM-based recommenders.

Limitations: Limitations vary across clusters in terms of scalability, precision, and safety. While Cluster 1 loses accuracy as user preference changes over time, its hybrid approach was mainly evaluated using limited data, instead of testing over large datasets. Cluster 2 struggles with overfitting and decreases accuracy when trying to increase fairness, and growth demands because computations' cost increase, moreover, over-smoothing reduces performance, while MacGNN may fail to capture nuanced behaviors. Rather than improving efficiency, current methods increase complexity. In contrast, Cluster 3's LLMs bring new risks, like vulnerability to prompt injections and hallucinations where it recommends items that are not available. Besides this, tools such as TokenRec need outside input, meaning updates must run separately whenever new users or products appear and a high computational cost for fine-tuning.

Susceptibility to Hallucination: The risk of hallucination differs across the clusters. Cluster 1 avoids it entirely, since the systems there only compare already known items, the same applies to Cluster 2, where models predict links among predefined nodes without generating anything new. On the other hand, Cluster 3 faces higher chances due to generative models producing false item names. Even when methods such as TokenRec aim to reduce errors by aligning choices with real entries, the core weakness still marks a key trait of LLM-driven solutions.

Table 1: Cluster Analysis Table

Cluster	**Method used**	**Strengths**	**Limitations**	**Hallucination**
Cluster 1: Traditional collaborative filtering approach for recommendation systems in e-commerce	CF and CBF are used. Specific methods include a scalable framework using Alternating Least Squares (ALS) matrix factorization in Apache Spark and hybrid systems merging CF with CBF	Scalable for big data. Highly efficient. High accuracy at 97% with hybrid method. Hybrid model addresses cold start problems. Spark model addresses scalability and data sparsity issues.	Accuracy suffers when user preferences are shifting (temporal dynamics). Hybrid model Tested on small dataset.	Not applicable. As they calculate similarity between existing items.

Cluster 2: Graph Neural Network Approaches in Recommendation Systems	GNN models were used, which take e-commerce data and models them as graphs and Macro recommendation graphs (MacGNN) which reduces graph size by clustering similar items and users into macro-nodes.	GNN can analyze the relationship between users and attributes easier than traditional methods. MacGNN successfully deployed on platforms and outperformed 12 baseline models. Increased fairness and reduced bias.	Scaling challenges and high computational cost. Over smoothing and challenges regarding efficiency and personalization. MacGNN clustering can oversimplify complex user behavior. Overfitting.	Not applicable. As they link prediction between existing fixed nodes without the ability to create new ones.
Cluster 3: Intersection of AI recommender systems in e-commerce to improve customer experience using LLM	LLM recommendation systems were used. A survey of LLM systems was conducted. TokenRec tokenizes user and item IDs using a masked vector-quantized tokenizer. The CheatAgent framework uses an LLM agent to generate prompt-injection attacks.	LLMs improve recommender systems and customer experience and gives better reasoning abilities than traditional methods. TokenRec outperforms traditional and existing LLM recommenders. TokenRec is highly efficient.	High cost of fine-tuning. Vulnerable to prompt-injection attacks. Vulnerabilities allow the system to recommend irrelevant items “hallucination. TokenRec is dependent on external information, requiring a separate update process for new user/items.	Vulnerable. As generative models can invent fake item titles, TokenRec tries to mitigate by mapping user preferences to existing items.

5.3 Scientific Gap

The analysis of the three clusters reveals a critical trade-off between recommendation quality and the system’s safety, with traditional methods used in both clusters 1 and 2 being immune to hallucination as they only rely on closed-set retrieval. However, their methods lack the reasoning to explain why the items fit the users’ needs, unlike cluster 3 that introduces generative recommendation by using LLMs, which solves the issue behind the lack of reasoning but introduces the risk of hallucination, which is the generation of products that fit the users’ needs, but are nonexistent or are out of stock, which significantly reduces the customer experience and affects how much they trust the system and that could deter them from using the platforms and result in customer churn. This is a more significant gap than the other limitations such as the cold start problem and high

computational cost, as the cold-start problem is mitigated by the hybrid approach and by LLMs. Moreover, it brings over a larger possibility of customer churn and loss of customers' trust than high computational costs.

Based on the analysis above, the scientific gap is
“Improving customer trust by mitigating hallucination in LLM-based recommender systems”

This gap is the primary issue with adopting LLM recommender systems, and it directly affects customer experience, as trust is the most important asset to a customer, and losing it means damaging the customer experience and could lead to losing the customer.

Research Question:

How can a recommender system be designed to take advantage of the reasoning and personalization capabilities of Large Language Models while guaranteeing that all outputs map to existing inventory items?

5.4 Thesis Objective

The objective of this thesis is to develop a recommender system that takes advantage of personalization and reasoning capabilities of LLMs while mitigating hallucination and making sure all outputs map to existing inventory items.

After conducting the literature review and identifying the scientific gap, research question and objective, the next chapter discusses the popular basic methodologies used in information system research to identify our solution approach.

6 Scientific Methodology

This chapter discusses the popular scientific methodology approaches and narrows down to exactly what methodology we will use in this paper in our solution approach.

6.1 Design Science

Design science is a research approach focused on designing solutions to help solve organizational problems, unlike traditional science that aims to explain what is true, design science aims to build an effective solution by building new artifacts, such as new models, systems or methods. The following subsections explore three influential frameworks, Hevner et al. (2004), Peffers et al. (2007) and Offermann et al. (2009).

6.1.1 Design Science in Information Systems Research (Hevner et al., 2004)

Hevner et al. (2004) lay out a foundational framework for evaluating design science research in the field of information systems. The authors present two complementary paradigms that define information systems research, first is the behavioral science paradigm, which aims to develop and verify theories relating to human and organizational behavior, second is the design science paradigm, which aims to create innovative artifacts that improve human and organizational capabilities (Hevner et al., 2004, p. 75). This paper proposes a framework that integrates both of these paradigms, this paper structures information systems research around three connected cycles, first is the relevance cycle that connects research to the application environment, second is the design cycle which drives the construction and evaluation of artifacts, and last is the rigor cycle that grounds research in the existing knowledge base of theories and methods, as represented in **Figure 6** which illustrates how the environment, the IS research process, and the knowledge base interrelate (Hevner et al., 2004, p. 80).

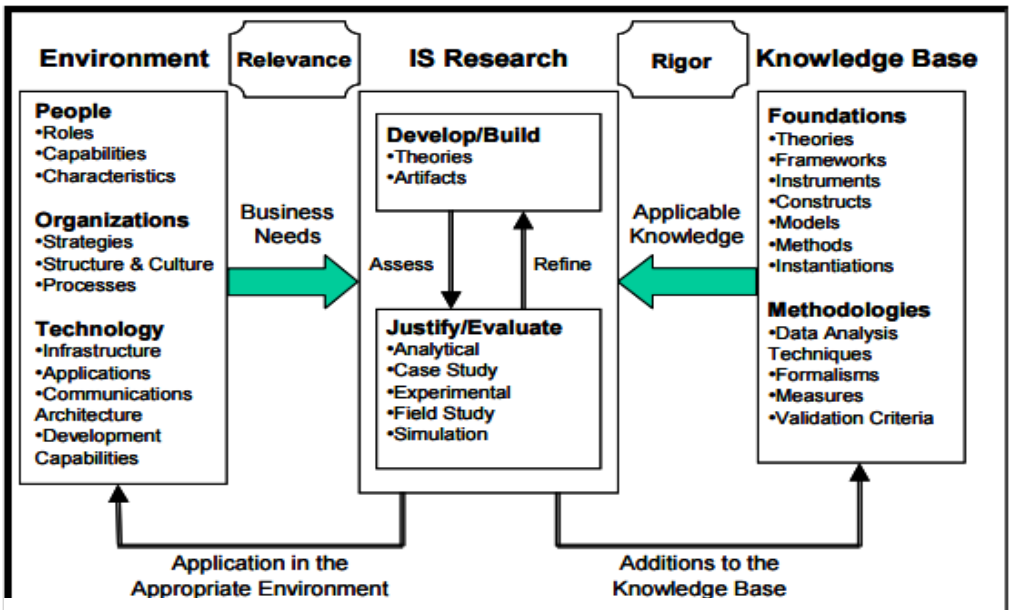

Figure 6: Information Systems Research Framework (Hevner et al., 2004)

A key contribution is the articulation of the seven guidelines of design science research, which include the production of a viable artifact, a problem that is relevant to important business needs, rigorous testing and evaluation, meaningful research contributions, methodological rigor, design as a goal directed search process and communication of findings to technical and managerial audiences (Hevner et al., 2004, p. 83–90).

In conclusion, Hevner et al. (2004) provides a reference for information systems research, creating a framework that is an applicable methodological framework for design science research.

6.1.2 A Design Science Research Methodology for Information Systems Research (Peffers et al., 2007)

Peffers et al. (2007) showcases a design science research methodology for information systems research, the authors argue that despite the growing recognition of design science, its use within information systems has remained slow due to the absence of a commonly accepted methodological framework, that provides both a process for conducting design science research and a model for presenting and evaluating the results (Peffers et al., 2007, p. 46).

This paper builds directly on the work of Hevner et al. (2004), using the seven guidelines as a starting point, then the authors present a six-phase design science process, first is problem identification and motivation, second phase is definition of objectives for a solution, third phase is design and development of the artifact, fourth phase is demonstration, fifth phase is evaluation and last phase is communication (Peffers et al., 2007, p. 52–55). The design science research model is visually represented in **Figure 7**, which shows the six phases.

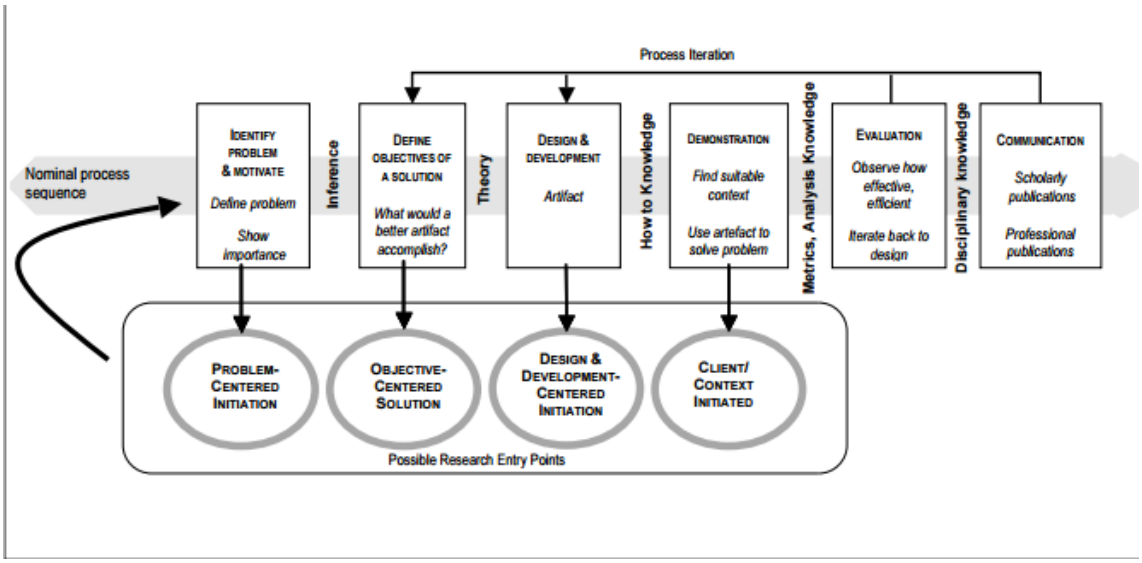

Figure 7: Design Science Research Model (Peffers et al., 2007)

Peffers et al. (2007) then applied this model to four published information systems research, first being the CATCH public health data warehouse, then a software deployed at MBA technologies, after that an SIP-based voice and video over IP application and finally a CSC method for information systems planning at Digia, with each of them aligning well with the six phases and having a different entry point, showing this methodology's flexibility (Peffers et al., 2007, p. 57–68). They then test the model against three objectives being consistent with previous design science, provision of a nominal research process and provision of a mental model for research outputs, they concluded that all three are satisfied (Peffers et al., 2007, p. 69).

In conclusion, Peffers et al. (2007) provides a useful methodology for design science information system research, which involves design, development and evaluation.

6.1.3 Outline of a Design Science Research Process (Offermann et al., 2009)

Offermann et al. (2009) argues that while individual research methods in information systems are well established, using only one method is not enough for producing rigorous results, Offermann et al. (2009) main contribution is a multi-method process for information system design science that combines both qualitative and quantitative methods into one step by step framework filling Hevner et al. (2004) and Peffers et al. (2007) gaps (Offermann et al., 2009, p. 1).

This paper introduces a process made up of three main phases, first being problem identification, then solution design and finally evaluation, with each of them being divided into smaller steps. The first phase of problem identification includes identifying the problem, conducting literature research, performing expert interviews and pre-evaluating relevance through general utility hypothesis in the form of that if this solution is applied an observed aspect will improve in a way that benefits other relevant items. After that the solution design phase centers on artifact design with a second round of literature research. Finally, the evaluation phase is detailed, comprising hypothesis refinement using a MECE (mutually exclusive, collectively exhaustive) approach, case studies or action research to test the practical application, surveys to assess relevance and experiments against existing solutions, this framework is represented in **Figure 8** (Offermann et al., 2009, p. 5–7).

This methodology is demonstrated through two case studies, the first one being the development of a service oriented architecture method and tool (SOAM) at Deutsche Telekom Laboratories and the other project is developing a business rules derivation and identification method, both cases follow the proposed process and show its adaptability across different information system design cases (Offermann et al., 2009, p. 7–10).

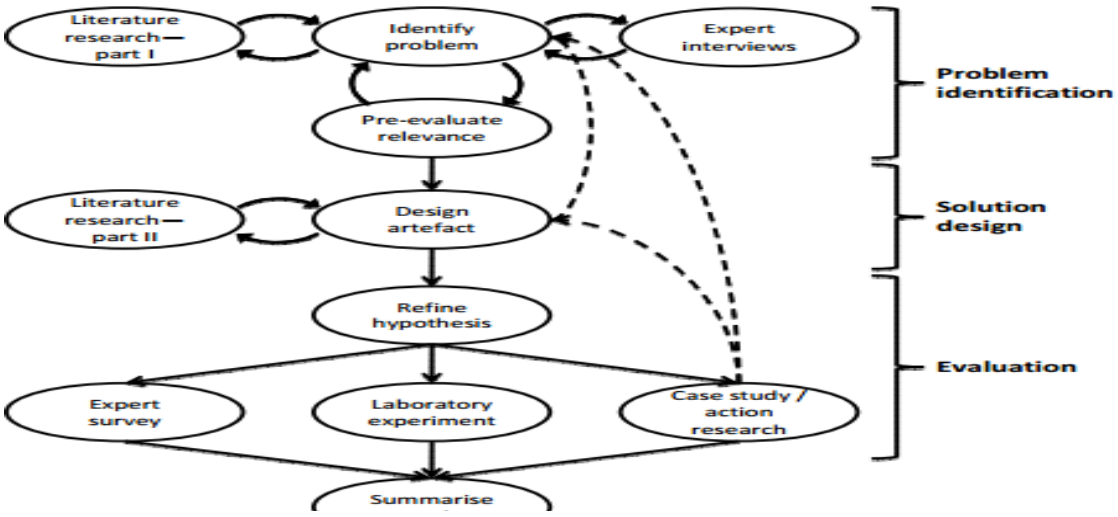

Figure 8: Proposed Research Process (Offermann et al., 2009)

In conclusion, Offermann et al. (2009) improved on the design science research methodology by providing additional step by step guidance that complements previous research by Hevner et al. (2004) and Peffers et al. (2007).

6.2 Qualitative and Quantitative Research

There are two main paradigms for research methodology that are mainly differentiated by the data they use and analyze. First is quantitative research that focuses on statistical and numerical data, then there is qualitative research that focuses on descriptive data (Bhangu et al., 2023, p. 40).

Qualitative research methods refer to techniques of investigation and depend on nonstatistical methods of data collection and analysis, providing a way to learn about non-quantitative data like languages, histories and people's experiences for example (Bhangu et al., 2023, p. 39). Qualitative research is mostly open-ended and allows the researchers to understand the object from multiple points and encourage the discovery of a reality not yet given by the methods used, making qualitative research suited for exploring contexts that cannot be reduced to numerical data (Bhangu et al., 2023, p. 41).

On the other hand, quantitative research commits to statistics, it relies on data collection and analysis based on a logical method with a focus on testing, its main features include data collection by standardized research instruments, finding large sample sizes and reporting results (Ghanad, 2023, p. 3794). This process moves from theory to hypothesis to statistical testing. Researchers use techniques like group comparisons to test the hypothesis (Ghanad, 2023, p. 3795).

Information systems research relies on both methods, as applied research relies on both, as mentioned by Bhangu et al. (2023, p. 41) that qualitative research methods are first used to identify the intersecting factor, then quantitative methods are used to measure and establish patterns between those factors.

6.3 The Delphi Method

The Delphi method is an old research method that evolved with time, with the modern application starting from around 1950 where it was used in the Cold War to help determine the possibility of enemy attacks, and since it has become a reliable method for studies to collect the opinion of a group for a complex problem and for future planning (Massaroli et al., 2017, p. 3). The main characteristic of this method is the consultation of experts on a specified topic and anonymity for all participants, which prevents a single participant's opinion from influencing the group and gives all participants the space to give and defend their opinions (Massaroli et al., 2017, p.5).

The Delphi method consists of four rounds, with the first round being a questionnaire with open questions, which allows the participants to express their opinions, with content analysis used on the resulting data, the second round is developed based on the first round's responses using quantitative questions and searching for agreement between

participants using statistical techniques, finally the third and fourth round are structured based on the responses of previous stages and follow the same steps of the second round, another version of the method exists with the first round being face to face interviews or focus groups, with the following rounds being like the regular method (Massaroli et al., 2017, p. 3).

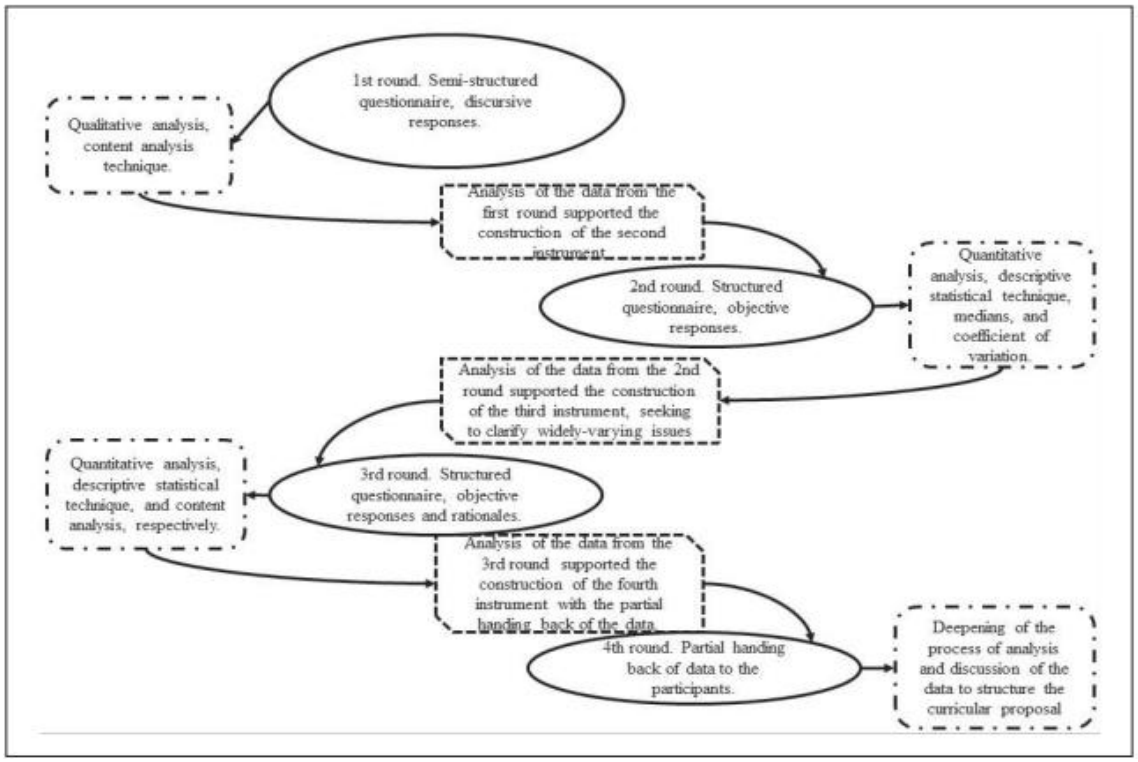

Figure 9: Delphi Method Rounds (Massaroli et al., 2017)

The Delphi method is a mixed method approach as it uses both qualitative and quantitative methods, with the qualitative approach allows for detailed information on experiences, emotions and values that influence the information, while the quantitative approach is used in measuring the data and comparing between variables, together both of these methods are used for investigations (Massaroli et al., 2017, p. 4). The literature advises 30 to 50 participants to discuss the topic and capture a mixture of opinions, and studies estimate the abstention rates being 20% to 50%, although higher retention rates have been observed when participants have a strong interest in the topic (Massaroli et al., 2017, p. 6). Data analysis takes place at the same time as data collection, with content analysis being used on qualitative data in the first round and statistical analysis being used on quantitative data from the other rounds (Massaroli et al., 2017, p. 7).

In conclusion, the Delphi method mixes qualitative and quantitative methods to provide the detail and subjectivity with objectivity and capacity, with its structure of interactive rounds and analysis (Massaroli et al., 2017, p. 8).

6.4 Knowledge Discovery in Databases (KDD)

Knowledge Discovery in Databases (KDD), referred to as data mining, has established

itself as an active and developing area in information technology, with growth in the data analyzed, knowledge discovered and techniques developed. Traditionally, KDD is an autonomous data driven trial and error process, whose task is to turn data into a story finding hidden information in a business issue, without human involvement, as much as it showed innovation, it has a limitation being that few of the patterns and algorithms published in research have actually been used in real business (Cao & Zhang, 2007, p. 1).

Traditional KDD treats data as the only objects for mining, using predefined models that the users cannot extend and evaluate the results based only on technical metrics like algorithm accuracy without the end user having an impactful role, which is more interesting to research rather than business. To address this gap, Cao and Zhang (2007) introduced domain driven data mining, which works with domain knowledge and domain experts to work together and tell the story and the end user accepts or rejects the mined results, the methodology consists of six components, domain-specific problem understanding, a constraint based mining context, in depth pattern discovery, a loop closed iterative refinement process, actionability of mined results and a human-machine-cooperated infrastructure (Cao & Zhang, 2007, p. 2-3). With the main goal being knowledge actionability, which is that the data is interesting to data miners and useful to decision makers.

The framework is operationalized through the DDID-PD process model, which covers problem understanding, constraints analysis, data preprocessing, modelling, in-depth mining, actionability enhancement, deployment and knowledge delivery, with the order of phases not being rigid as any step could be revisited by the domain experts (Cao & Zhang, 2007, p. 8–9).

6.5 CRISP-DM

Cross Industry Standard Process for Data Mining (CRISP-DM) is the standard process model for applying data mining projects, despite being released in 2000, it remains widely used in both research and industry (Schröer et al., 2021, p. 526). This relevance is due to this model being easy, reliable, structured and commonly used. It is used in multiple industries such as health and education, engineering, governments and information technology (Schröer et al., 2021, p. 529).

CRISP-DM is made up of six phases that guide a data mining project from business understanding to deployment. The first phase, business understanding, is determining the data mining goal such as the mining type and producing a project plan, the second phase, data understanding is collecting data from sources and exploring and describing it, while checking its quality, then the third phase, data preparation, covers data cleaning and transformation, after that the fourth phase is modelling, which starts by selecting a modelling technique, and testing it then comparing against defined criteria, the fifth phase, evaluation, compares the final results to the business objectives, with most studies using quantitative metrics and visualizations like trees and confusion matrices, the final phase is deployment, which could be a final report or software that then needs planning,

monitoring and maintenance (Schröer et al., 2021, p. 527).

Regardless of the strengths of CRISP-DM, a repeated problem is that most reviewed studies disregard the deployment phase, despite it being defined in the model, Schröer et al. (2021) suggests two explanations being that the focus of most studies is just building and evaluating models, and that there are missing guidelines for conducting the deployment phase, which indicates that CRISP-DM does not cover the deployment phase quite well (Schröer et al., 2021, p. 531). Multiple improvements to CRISP-DM have been proposed for this limitation, such as the APREP-DM framework that extends CRISP-DM with pre-processing tasks for sensor data analysis, and the Lean Design Thinking Methodology (LDTM) which adds design thinking into CRISP-DM to better manage modern technologies like machine learning algorithms, despite these efforts, none of them has yet been significantly adopted and CRISP-DM remains widely used (Schröer et al., 2021, p. 532).

This chapter discusses the basic popular methodologies, with the aim being to derive our own solution approach from them, which is presented in the next chapter.

7 Solution Approach

After looking at the general methodology approaches, we created our solution approach to address the scientific gap of mitigating hallucination in LLM-based recommender systems in e-commerce, following the design science research process of Offermann et al. (2009), this solution approach is structured into three sequential phases, the first phase is problem identification, then the second phase is design and the final phase is evaluation, each phase is based upon the outcomes of the previous phase, meaning that the phases have to be conducted in order.

Phase 1-Problem Identification: This phase is the start and foundation of this research, it starts with a structured literature review, in which state-of-the-art academic papers relating to the research are collected using a specific set of selection criteria, these papers are then grouped into clusters based on the technologies used or what the paper discusses, they are then analyzed thoroughly and comparatively, the goal of this analysis is to identify a scientific gap and form a research question, establishing the objective of this paper and using this gap in the design phase.

Phase 2-Design: The design phase begins with the analysis of existing partial solution of the identified scientific gap, after the analysis is done functional requirements, non-functional requirements and components of these partial solutions are extracted, these extractions are then used to design the proposed framework, which is then described in both structural and behavioral terms using models, the models are then used to inform our implementation of this solution in the next phase, this process is iterative and has to be done in order to design an acceptable solution.

Phase 3-Evaluation: This phase begins with describing the used tools, implementing the solution and the showcasing of a working demo of the proposed solution to confirm it has

met the design phase requirements, its then evaluated quantitatively against the design requirements to ensure that all objectives are met and the solution works up to par, after that, qualitative expert interviews are conducted to collect feedback from field experts or academics, this feedback gathered is then used to refine the solution further.

Figure 10 demonstrates the complete solution approach in detail as an illustration, in the next chapter, this solution approach will be applied to create the solution.

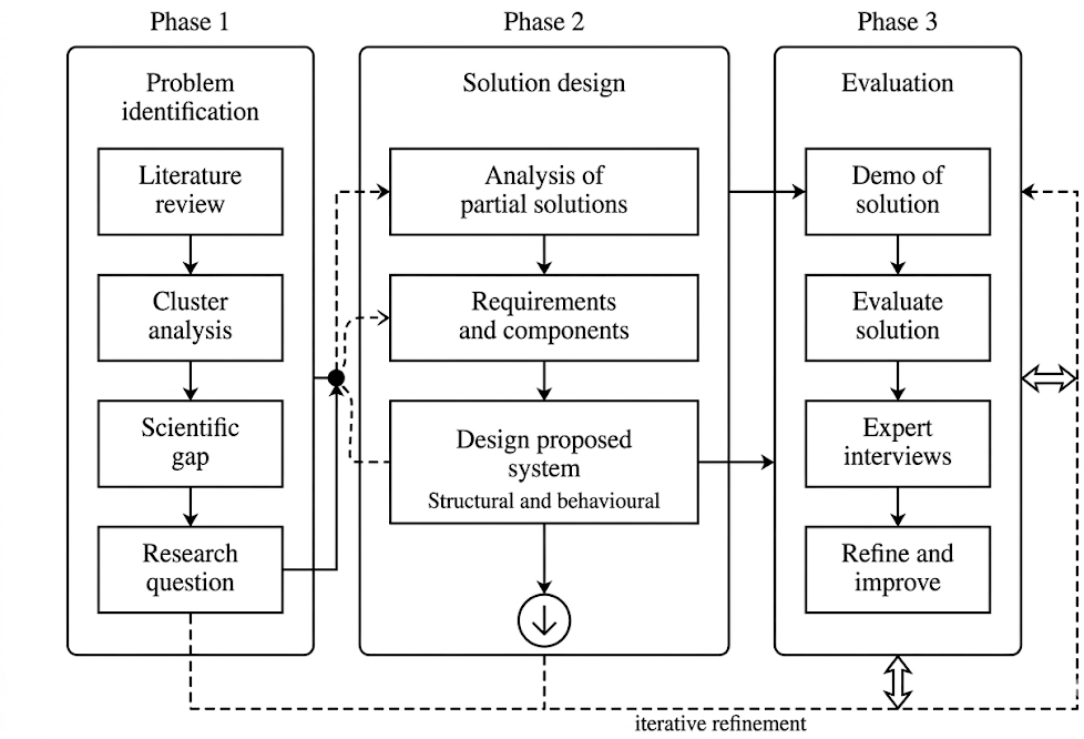

Figure 10: Solution Approach Overview

8 Results

In this chapter, we will apply our solution approach demonstrated in the previous chapter, following the same iterative sequence of the three phases, problem identification, design and evaluation.

8.1 Problem Identification

This phase has already been done in chapter 5 Literature Review, which followed the sequence of gathering state-of-the-art papers according to a certain selection criterion, and then clustering them and analyzing all 3 clusters to identify our scientific gap "Improving customer trust by mitigating hallucination in LLM-based recommender systems", research question and the objective to build a multi-agent AI recommendation system that mitigates hallucination while maintaining the reasoning capabilities of LLMs

and making sure all results map to existing inventory items, which is essential for the design phase to be able to deliver an optimal solution.

8.2 Design

Following the problem identification phase, this design phase presents an analysis of existing solutions that address the same scientific gap. The objective of this analysis is not like the literature review, rather it is to examine solutions for our scientific gap and to extract requirements from partial solutions.

8.2.1 Analysis

The four selected papers are separated into two clusters, with the first examining LLM-powered agentic recommendation systems, and the second examining RAG-based hallucination mitigation frameworks.

8.2.1.1 Cluster 1: LLM-Powered Agentic Recommender Systems

This cluster examines papers that explore the use of agentic AI in recommender systems, where hallucinations are addressed using tools and structured planning.

RecMind: Large Language Model Powered Agent for Recommendation (Wang et al., 2024)

Wang et al. (2024) introduced RecMind, an autonomous LLM agent designed for recommendations, RecMind addresses a limitation of recommender systems, which is their inability to use external knowledge due to dataset specific tuning. Instead of training the model only on recommendation data, RecMind treats recommendation as an agent task, where an LLM reasons and interacts with external tools like SQL queries and search APIs to get structured and verifiable information from a memory component. A notable contribution of this paper is the self-inspiring planning algorithm, where the agent considers all previously explored paths before committing to an action at every step (Wang et al., 2024, p. 4351-4353).

RecMind was evaluated using five recommendation scenarios such as rating prediction, sequential recommendation, and recommendation on the Amazon reviews dataset, outperforming existing LLM baselines and achieving performance competing with the fully pre-trained model P5 (Wang et al., 2024, p. 4355).

Limitations of this paper include the use of ChatGPT backend, making it sensitive to privacy issues or offline deployment, moreover, tools reduce hallucinations, but it does not map recommendations to a certain product catalogue, therefore, introducing a hallucination risk (Wang et al., 2024, p. 4359).

A Hybrid Multi-Agent Conversational Recommender System with LLM and Search Engine in E-commerce (Nie et al., 2024)

Nie et al. (2024) introduce a hybrid multi-agent conversational recommender system

designed for e-commerce use only, this solution addresses the issue of system latency and hallucination due to the changing supply chains, the authors tried to mitigate this problem by pairing a 13 billion parameter LLM model (ChatSR) with a dedicated search agent, this puts the burden of retrieval on an external infrastructure, therefore, limiting the output to available inventory (Nie et al., 2024, p. 746).

The system operates by making ChatSR handle the user interactions and generate search queries, the search agent then processes these queries and retrieves relevant items then returns the output, if the LLM generates irrelevant queries or items are out of stock, the agent adjusts by giving substitutes, this process of differentiating between the process of LLM reasoning and retrieval by the search agent mitigates hallucinations (Nie et al., 2024, p. 746).

Despite its good performance, this system still has its limitations for generalized or privacy sensitive deployment, it also lacks agents for complex analytical queries over tabular data (Nie et al., 2024, p. 746).

8.2.1.2 Cluster 2: RAG-Based Hallucination Mitigation

This cluster focuses on papers that explore the use of RAG based recommender systems to mitigate hallucination, this cluster provides theoretical grounding on how RAG based architectures reduce hallucinations and their limitations.

Hallucination Mitigation for Retrieval-Augmented Large Language Models: A Review (Zhang & Zhang, 2025)

Zhang and Zhang (2025) present a review of what causes hallucinations in RAG architectures, despite acknowledging that RAG reduces hallucinations significantly by using domain-specific retrieval, it still has limitations, the review categorizes these errors into two stages, first being retrieval failures which are the issues at the data extraction level, like outdated data sources, unspecified user queries and sub-optimal retrieval granularities that feed noisy context to the model, the other errors occur at the generation stage, as LLMs can incorrectly handle context, leading to noise and contradictions between retrieved knowledge and the model's memory (Zhang & Zhang, 2025, p. 9-12).

Moreover, Zhang and Zhang shine light on the structural limitations in standard RAG systems, like the "middle curse" where LLMs show attention loss and fail to correctly use information in long retrieved contexts. To mitigate this, the authors advise for use of other techniques with RAG, like knowledge graphs (Zhang & Zhang, 2025, p.13-14).

Mitigating Hallucination in Large Language Models (LLMS): An Application-Oriented Survey on RAG, Reasoning, and Agentic Systems (Li et al., 2025)

Li et al. (2025) introduce a taxonomy that differentiates between knowledge-based hallucinations, like creating a non-existing item, and logic-based hallucinations, which have flaws in reasoning despite having all the correct factual premises. This survey

evaluates how different architectures handle failures and argues that RAG only systems solve knowledge-based problems but cannot solve logic-based hallucinations (Li et al., 2025, p. 12, 15).

Li et al. (2025) examine the intersection of RAG and reasoning enhancements, they suggest that combining the knowledge grounding of RAG with multi-step logical planning forms “Agentic Systems” that help mitigate hallucinations. However, the survey notes that current agentic frameworks face challenges regarding risk of errors in multi-agent pipelines, proposing the use of unified and lightweight designs to balance efficiency and accuracy (Li et al., 2025, p. 15-17).

8.2.1.3 Discussion

This analysis section discusses four papers separated into two clusters to find the elements needed for the solution.

Both clusters address hallucination, the first cluster focuses on mitigating hallucination by routing retrieval to agents and tools, while the second cluster focuses on RAG and demonstrates how systems using RAG reduce hallucination but can face logical problems, which could be fixed by a combination of RAG and a reasoning system for full mitigation, the four papers together demonstrated that using a multi-agent LLM system with verified retrieval is a good approach.

All four papers show gaps, with RecMind and Nie et al’s system both struggling with analytical queries like price filtering, category comparisons or statistical queries, which is important for e-commerce solutions. Zhang & Zhang (2025) acknowledge that even RAG systems suffer from context conflict issues, and Li et al. (2025) shows that agentic systems could struggle with error propagation.

Moving towards our solution, this analysis shows that no single solution fully addresses the gap and shows that RAG knowledge grounding must be paired with reasoning to reduce both types of hallucination, these solutions collectively inform the design of our proposed solution.

8.2.2 Requirements

This chapter shows the extracted requirements from the analysis in the previous chapter. These extractions help to inform us of the design of our proposed solution.

8.2.2.1 Functional Requirements

Functional Requirements define what the solution must be able to do. Based on the analysis, these functional requirements are identified.

Functional Requirement 1: NLP: The solution must be able to accept and understand language queries from users, as identified in RecMind and Nie et al. (2024)’s solutions.

Functional Requirement 2: Grounded Product Search: The solution must retrieve product recommendation data exclusively from a verified dataset, ensuring all recommendations map to existing inventory items, this requirement is motivated by the hallucination mitigation strategy of Nie et al. (2024) and the retrieval principles mentioned by Zhang & Zhang (2025).

Functional Requirement 3: Analytical Query Handling: The solution must be able to handle analytical queries over tabular data, such as price filtering, category comparisons and statistical queries, this requirement is motivated by the limitations of cluster 1.
Functional Requirement 4: Query Routing: The solution must be able to classify user queries and route them to the appropriate agent or tool, differentiating between search and analytical queries, as outlined in the multi-agent architectures in RecMind and Nie et al. (2024).

Functional Requirement 5: Conversational Ability: The solution must be able to communicate naturally and summarize retrieved answers, as demonstrated across all reviewed systems.

8.2.2.2 Non-Functional Requirements

Non-functional requirements define the quality attributes the system must satisfy.

Non-Functional Requirement 1: Hallucination Resistance: The solution must ground all generated outputs to information from the product knowledge base, ensuring that no non-existent or fake product descriptions are present in the output, this is the primary non-functional requirement motivated by the scientific gap and the four partial solutions.

Non-Functional Requirement 2: Local Deployability: This system must be operational without the need to rely on cloud-based LLM APIs to address the privacy and offline limitations of RecMind.

Non-Functional Requirement 3: Modularity: Each agent must operate independently and can be updated or removed without affecting the whole solution using modular architecture, as recommended by Li et al. (2025) to have lightweight agentic design to reduce the risk of error propagation.

After extracting our functional and non-functional requirements, the next subchapter applies them to create our solution framework, which is divided into structural and behavioral models that demonstrate the components of this framework and how the execution process works.

8.2.3 Conceptual Model

This chapter presents the design of our proposed system, which is developed from the

functional and non-functional requirements extracted in the previous chapter, the solution is presented in two complementary models, first is the structural model, which defines the architectural components of the system and relationships without the order of execution, next is the behavioral model, which describes how these components interact with each other and it what order during execution, showing the flow of execution from start to finish.

8.2.3.1 Structural Model

The structural model presents the solution as a set of four connected components, with each of them having their own functionality. These components together form a multi-agent architecture in which the LLM reasoning is decoupled from the retrieval, ensuring that the system outputs are grounded in the product base and hallucination is constrained as shown in **Figure 11**.

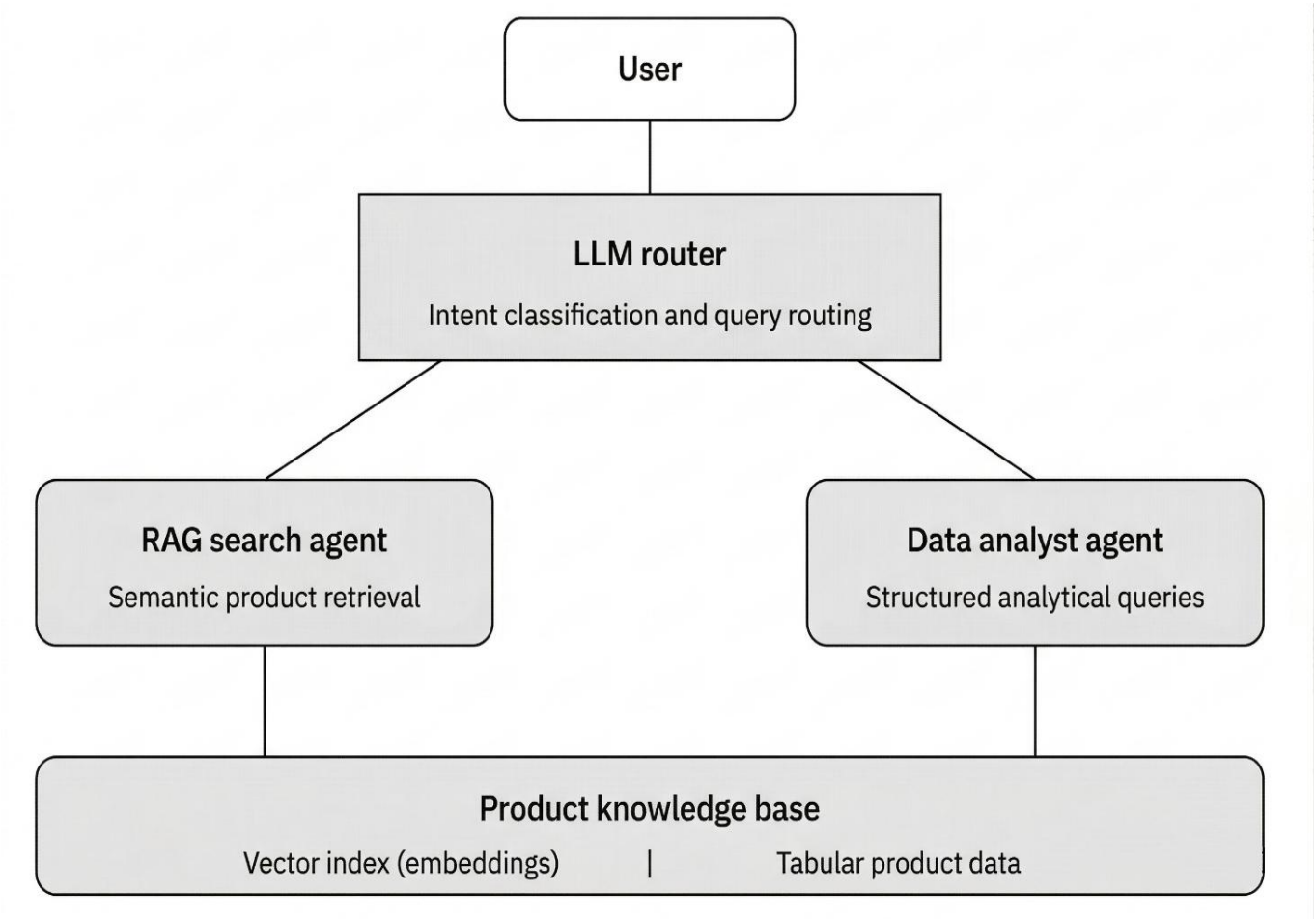

Figure 11: Structural Model

This structural model is made up of four main components of our solution, which is a multi-agent LLM recommender with two agents, the first being a search agent using RAG to ground the retrieval process to a specified embedded product knowledge base using vector indexing, and the second agent is a data analyst agent for structured analytical queries using the same product knowledge base but as a tabular form.

LLM Router

The first component is the LLM router, which uses an LLM model for its operations and is the reasoning core of the framework. This router receives the user's natural language query and then classifies its intent before redirecting it to the appropriate agent, it routes to the RAG search agent for semantic queries like product recommendation or the data analyst agent for structured queries like price comparisons, it also formats the responses returned from the agents into natural language. The router does not perform the retrieval itself, making sure that the generative capabilities are grounded to reasoning and routing only, rather than product generation, as informed by the central agent pattern by Nie et al. (2024) and planning component of RecMind (Wang et al., 2024).

RAG Search Agent

The RAG Search Agent, which is responsible for all the product search tasks, after receiving a routed query from the LLM router, it encodes it using the embedding model, performs semantic similarity search over the vector index within the product knowledge base and then returns a list of the product results, due to all the outputs being retrieved from the knowledge base rather than generated, this agent eliminates knowledge-based hallucinations for search queries, as outlined by Zhang & Zhang (2025).

Data Analyst Agent

The Data Analyst Agent handles all analytical queries, it uses the Pandas library to operate directly over the tabular product database, enabling price filtering, category comparisons and ranked result generation. This component addresses the cluster 1 Gap, where no solution provided structured data analysis capabilities alongside conversational search. It also serves to mitigate hallucinations as it goes over the same product knowledge base of the RAG search agent but with different formatting, eliminating the possibility of retrieving products that do not exist in the knowledge base.

Product Knowledge Base

The Product Knowledge Base is the ground truth for all the system outputs, it consists of two representations of the same product dataset, a vector index of embedded product data done by an embedding model, this embedded vector index is used by the RAG search Agent and a structured tabular file used by the Data Analyst Agent. By making sure that both agents retrieve data from the same verified dataset, this architecture ensures that no output can reference a product that does not exist in the inventory, satisfying the non-functional requirement that hallucination should be mitigated.

The next subchapter shows the behavioral model, which presents how all the components shown in the structural model interact with each other and in what order.

8.2.3.2 Behavioral Model

The behavioral model describes the execution of the proposed system across six stages, tracing the lifecycle starting from the user input through the routing and agent selection up until the response presented to the user as shown in **Figure 12**.

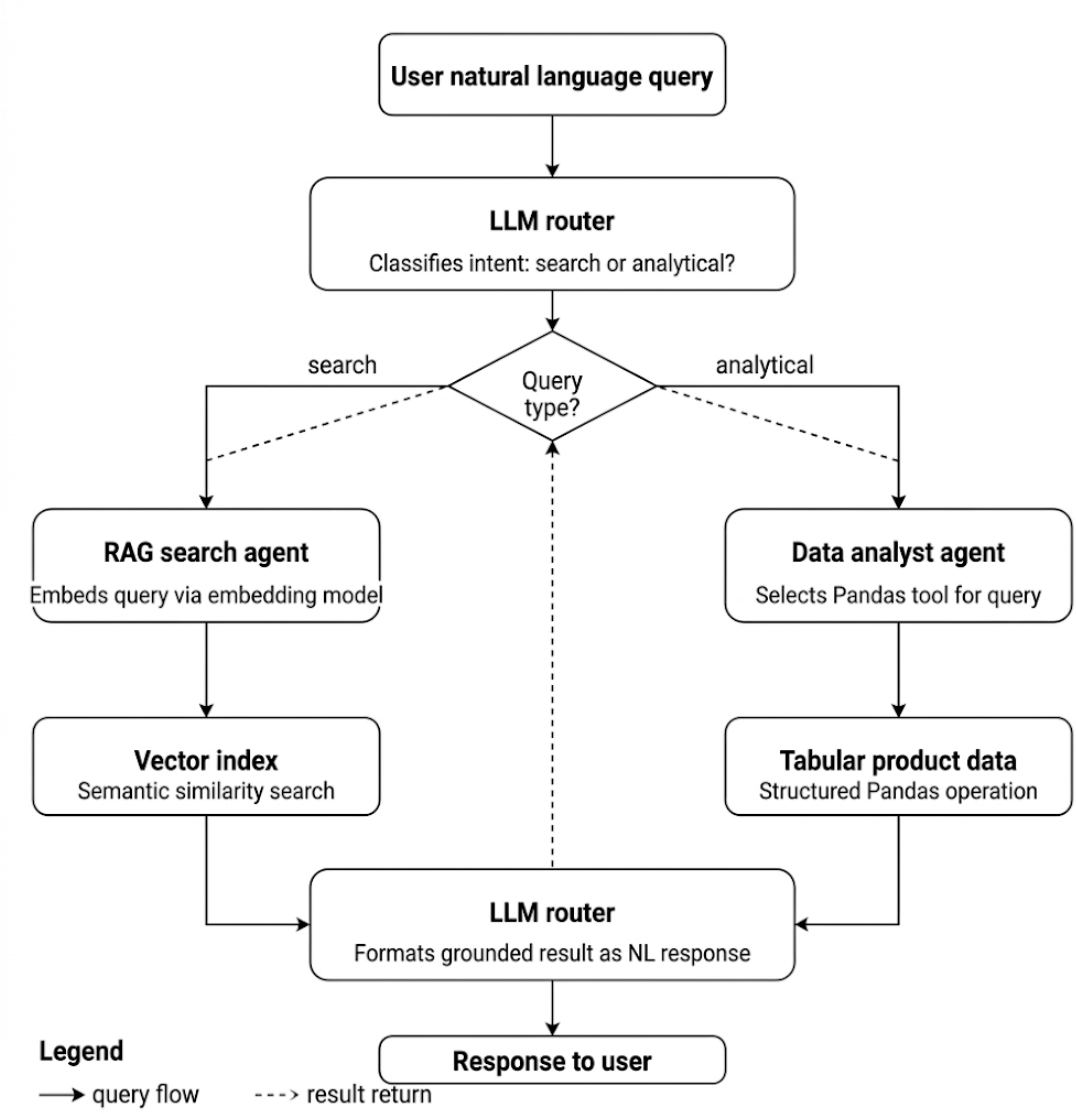

Figure 12: Behavioral Model

Step 1: User Query

The framework starts off with the user submitting a natural language query requesting anything related to the product category through the conversational interface, which could be descriptive searches or analytical searches, which are then passed to the LLM router.

Step 2: LLM Router

After the user inputs their query and it gets passed to the LLM router, the router processes this query and identifies its intent, then it is classified into one of two categories, either search-oriented, where the user wants product recommendations or descriptions, or

analytical, where the user is seeking structured insights like price comparisons or category statistics. The router also detects regular conversational queries and responds by itself without invoking an agent. This step uses LLM reasoning for routing but does not generate products by itself.

Step 3: Agents

Based on this classification, the user query is routed to the needed agent, search queries are directed to the RAG search agent, which encodes the query using the embedding model and perform semantic similarity search over the vector index inside the product knowledge base, analytical queries are routed to the data analyst agent, which uses the needed tools from the Pandas library and executes them over the tabular product database.

Step 4: Retrieval

After the agent receives the appropriate query, depending on the query, each agent retrieves the results exclusively from the product knowledge base, with the RAG search agent retrieving from the embedded database using cosine similarity and the data analyst agent retrieving from the tabular database using Pandas operations, ensuring that no product output is generated from the LLMs knowledge, this is critical for mitigating hallucinations, as agents can only return products that exist within the verified dataset.

Step 5: Formatting

After the data is retrieved, the results are then returned to the LLM router, which formats them into a natural language response that fits the conversation. The LLM in this stage is only limited to language formatting rather than knowledge generation, enforcing the point to mitigate hallucination.

Step 6: Response

Finally, the result is returned to the user in a natural language, with the user having no input over the routing or retrieval processes.

This two-tier architecture where the reasoning and retrieval are separated and all outputs are verified in our knowledge base, satisfies all our functional and non-functional requirements.

Figure 13 shows a sequence diagram that emphasizes the process of the proposed multi-agent RAG recommendation system, showing how the process moves sequentially from the user query to retrieval up to the LLM router formatting the response in a natural language to be returned to the user.

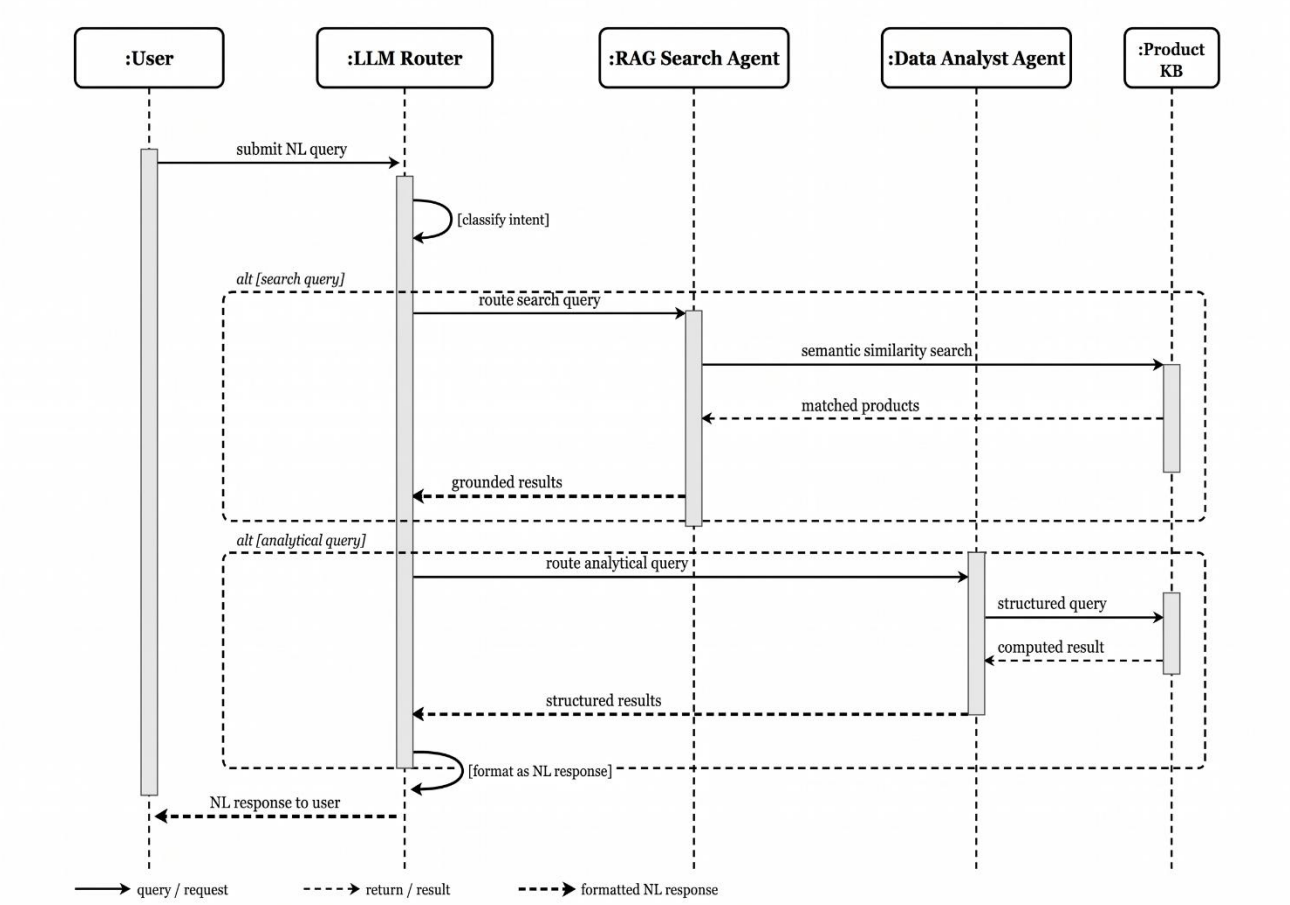

Figure 13: Sequence Diagram for The Multi-Agent AI recommendation and analysis system

After we analyzed partial solutions and extracted the necessary functional and non-functional requirements and then designed our behavioral and structural model, the next chapter we implement, demonstrate and evaluate our solution based on our requirements and models.

9 Evaluation

After designing our solution following the solution approach, this chapter presents the evaluation phase as outlined in the solution approach, this phase's goal is to implement our proposed solution and then assess whether the designed framework satisfies the functional and non-functional requirements extracted in the design phase, starting with describing all tools and datasets that were used for the design of our solution and then the installation process of all tools used and after that the framework demonstration that gives an overview of the system and confirms it works as designed, after that, the framework is represented through interactive scenarios that shows how the system handles different query types. Finally, qualitative expert interviews are conducted to gather feedback, and the results of these interviews will lead to a final discussion of the system's strengths, limitations and directions for improvement.

9.1 Tools Used

The implementation of our proposed solution is a locally deployed, RAG, multi-agent LLM recommendation system that uses multiple open-source tools, each of those tools was chosen with a specific purpose to deliver on our functional and non-functional requirements extracted in the previous design phase, all the tools mentioned run locally and free without the use of any cloud tools or subscriptions.

The framework uses Ollama for the execution of LLMs locally, it is an open-source tool that manages model downloading, quantization and hardware allocation. The selected model for this framework is Qwen3.5:4b, released in March of 2026, this is a state-of-the-art model chosen for its great performance in benchmarks and tests, our framework uses this model with the thinking mode disabled to reduce latency, the same model is used for routing to the different agents and responses for the agents.

For our product knowledge base and vector retrieval, ChromaDB is used, which is a lightweight, embedding database designed for RAG applications and integrates with Python. An embedding model is needed to vectorize the dataset for ChromaDB, nomic-embed-text-v2-moe is used for this model, which is a multilingual text embedding model chosen for its excellence at retrieval. This model converts the product into vectors in ChromaDB and then are retrieved via similarity when using the RAG search agent.

The data analyst agent uses Python's Pandas library for structured queries, the agent translates the query into Pandas operations, and they are then performed on the CSV tabular data also stored in the framework, helping with queries like price filtering.
The UI is developed using Streamlit, an open-source Python framework used for developing web applications, it has native support for chat interfaces making it perfect for this use case.
The product dataset used in this framework is from Kaggle, it is a dataset for an Indian e-commerce website named Flipkart with 20,000 entries, this dataset was cleaned to remove any unnecessary categories or null values that could interfere with our solution, leaving important categories only like price, rating and product category, the cleaning process will be discussed in more detail in the next subchapter.

9.2 Dataset Preparation

As mentioned in the previous chapter, the product dataset is sourced from Kaggle, the dataset is an e-commerce product base with 20,000 entries from an Indian e-commerce website Flipkart, the raw dataset had 14 columns such as product identifiers, names, descriptions, categories, brand names, prices, product ratings, image links, and specifications.

Before embedding the database, preprocessing steps were done to make sure the data is compatible with the RAG pipeline and analytics layer. Firstly, there were 78 null values in the price fields which were replaced by the mean value of their categories as this

preserves the relative price distribution of that category, after that, missing values in the brand category were replaced with “Generic”, and 2 missing rows were deleted due to being empty in all columns leaving 19,998 records. Multiple columns that had no value for retrieval were removed like product_url, pid, is_FK_Advantage_product and crawl_timestamp, leaving the dataset with a total of 7 columns instead of 14.

After cleaning the dataset, two feature engineering steps were done to make it easier for retrieval, the product_category_tree field was stored as a nested string, was then parsed as one attribute to make it easier for retrieval, moreover, the product_specifications column was written as encoded dictionary, it was then turned into natural language paragraphs for retrieval.

9.3 Installation Process of Used Tools

This subsection documents the installation steps used to deploy our framework on a local machine. Starting with the Python environment, Ollama runtime and models, Python dependencies and knowledge base integration. This process was done on a Windows laptop.

The first step is to install Python through the official distributor, after that, a virtual environment is created in the project file to isolate the project dependencies from the whole system. The virtual environment is created with the following commands.

```
python -m venv venv
```

```
venv\Scripts\activate
```

After activation, all package installations are isolated in this environment.

The second step is installing Ollama through the official website ollama.com, after installation it can be accessed through the ollama command. After the installation of Ollama, the two used models are then pulled from the Ollama registry using the two following commands.

```
ollama pull qwen3.5:4b
```

```
ollama pull nomic-embed-text-v2-moe
```

The first command pulls the LLM model Qwen 3.5 4b which is used for routing, intent classification and response formatting. The second command pulls the embedding model nomic-embed-text-v2-moe which is used by the RAG agent and the knowledge base to generate vector representations of product descriptions.

The third step is installing all Python dependencies that are specified in a requirements.txt file in the project’s file, they are then installed with the following command.

```
pip install -r requirements.txt
```

The primary packages installed include ChromaDB, Ollama, Pandas and Streamlit.

The final step before running our framework is ingesting the dataset into ChromaDB vector store. This is a one-time operation that must be done so that the RAG search agent can retrieve the products from this dataset, this is done by running the knowledge base file in the terminal.

python knowledge_base.py

After that the framework can be run using the following command in the terminal.

streamlit run app.py

9.4 Framework Demonstration

This subchapter demonstrates the designed framework, starting with the starting page up to how each component works and operates across different interaction scenarios with the user.

Upon launching the web application, the user is presented with a conversational chat interface where the user can submit natural language queries built on Streamlit named Kart as it works on a Flipkart dataset, the system is able to handle two different query categories by two different agents. The first being search oriented queries where the user is looking for product recommendations or product descriptions, these queries are handled by the RAG search agent which will be covered in more detail in the next chapters. The second type of queries are analytical queries, where the user is looking for structured insights like price comparisons, statistics or filtered product rankings, these queries are handled by the Data analyst agent. The LLM router classifies each incoming query and routes it to the appropriate agent without requiring explicit direction from the user, making this multi-agent architecture transparent at the interface level.

Figure 14 presents a look at the system upon launch, showing the chat field and response area and recommended questions.

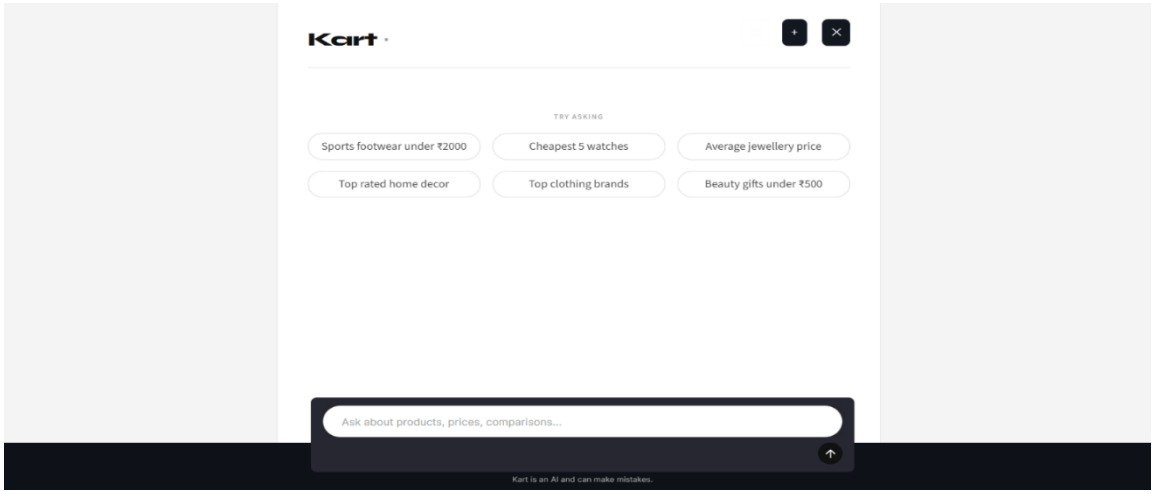

Figure 14: Framework Interface

Query Routing

The starting point after a user sends a query is the LLM router as shown in the behavioral model, after the user sends a query the router classifies its intent and sends it to the needed agent.

The router classifies the intent of the user by a system prompt that tells the LLM to reply with either “search”, “analytical” or “chitchat”, moreover the classify function includes recent conversation history so that follow up queries can be routed correctly as shown in **Figure 15.**

```python
_CLASSIFY_SYSTEM = (
    "You are a query classifier for an e-commerce assistant. "
    "Reply with exactly one word: 'search', 'analytical', or 'chitchat'. Nothing else."
)
def _classify(query: str, history: list[dict]) -> str:
    """
    Returns 'search' or 'analytical'.
    History is included so follow-up queries ("show me cheaper ones") route correctly.
    """
    prompt = _CLASSIFY_USER.format(
        history_text=_history_to_text(history),
        query=query,
    )

    response = ollama.chat(
        model=LLM_MODEL,
        messages=[
            {"role": "system", "content": _CLASSIFY_SYSTEM},
            {"role": "user",   "content": prompt},
        ],
        think=False,
        options={"temperature": 0},
    )

    label = response.message.content.strip().lower()
    if "analytical" in label:
        return "analytical"
    if "chitchat" in label:
        return "chitchat"
    return "search"
```

Figure 15: Router Classifying Code

Once the router classifies the intent, it follows one of three paths, the first being chitchat like greetings, where the LLM responds conversationally without passing the query to any agent, the other being search queries where the RAG search agent is called, for analytical queries, the router passes it to the data analyst agent, in both agents the results are passed back to the LLM to format the results in a natural conversational response and send it back to the user without the LLM introducing any products by itself as shown in **Figure 16.**

```
if intent == "analytical":
    result       = run_analyst_agent(query)
    results_text = format_analyst_results(result)
else:
    products     = run_rag_agent(query)
    results_text = format_rag_results(products)
messages = [{"role": "system", "content": LLM_SYSTEM_PROMPT}]
messages += _history_to_messages(history)
messages.append({
    "role": "user",
    "content": _FORMAT_USER.format(query=query, results=results_text),
})

response = ollama.chat(
    model=LLM_MODEL,
    messages=messages,
    think=False,
    options={"temperature": 0.3},
)
```

Figure 16: Routing and Formatting Code

Product Knowledge Base

The product knowledge base was built through a one-time process that converts the CSV dataset into vector representations in ChromaDB. For each product, a text is constructed combining the product name, description and specifications into one string, which is used as input to the embedding model as shown in **Figure 17.**

```
combined_text = (
    f"Product: {row['product_name']}\n"
    f"Category: {row['main_category']}\n"
    f"Brand: {row['brand']}\n"
    f"Description: {row['description']}\n"
    f"Specs: {row['specs_text']}"
)
```

Figure 17: Combining Text Code

Each product's data is stored alongside the vector in ChromaDB, allowing filtering during retrieval. The embedding is done in batches of 25 products at a time to manage memory usage, the collection is then configured with cosine similarity as the distance, so that during retrieval the lowest distance score products are the most like the user's request and are retrieved.

RAG Search Agent

The RAG search agent handles semantic queries such as recommendations or budget queries, when the agent is invoked, it first parses any price amounts and currency from the query, these demands like “below 900” are then used as filters in the following retrieval step as shown in **Figure 18.**

```
def _extract_price_limits(query: str) -> tuple[float | None, float | No
    q = query.lower()
    price_max = None
    price_min = None

    m = re.search(
        r'(?:under|below|less than|cheaper than|upto|up to|max)\s*'
        r'(?:rs\.?|inr|₹)?\s*(\d[\d,]*)', q
    )
    if m:
        price_max = float(m.group(1).replace(",", ""))

    m = re.search(
        r'(?:above|over|more than|at least|min)\s*'
        r'(?:rs\.?|inr|₹)?\s*(\d[\d,]*)', q
    )
    if m:
        price_min = float(m.group(1).replace(",", ""))

    return price_min, price_max
```

Figure 18: RAG Filter Code

The search is then done on ChromaDB, first, the query text is embedded using the same model using Ollama’s embedding API. The resulting vector is then compared to all stored products using cosine similarity, if there is no price filter a small product count is enough, but with a price filter more products must be looked at as many similar products may fall out of the price filter as shown in **Figure 19.**

```
def search_products(query: str, n_results: int = TOP_K_RESULTS) -> list[dict]:
    collection = get_collection()
    query_embedding = embed_batch([query])[0]

    results = collection.query(
        query_embeddings=[query_embedding],
        n_results=n_results,
        include=["documents", "metadatas", "distances"]
    )
```

Figure 19: RAG Search Code

After retrieval, if the semantic search returns fewer products than expected, the agent looks at the CSV file using Pandas to deliver the needed amount, and since the CSV has the same products, this ensures no generated products that are not available in the product knowledge base, after that, the results are then formatted into a natural language response to be presented to the user.

Figure 20 shows the RAG search agent responding to a query ““I am looking for good black shoes for my son to wear them to school under 800”, showing the combination of semantic search and price filtering.

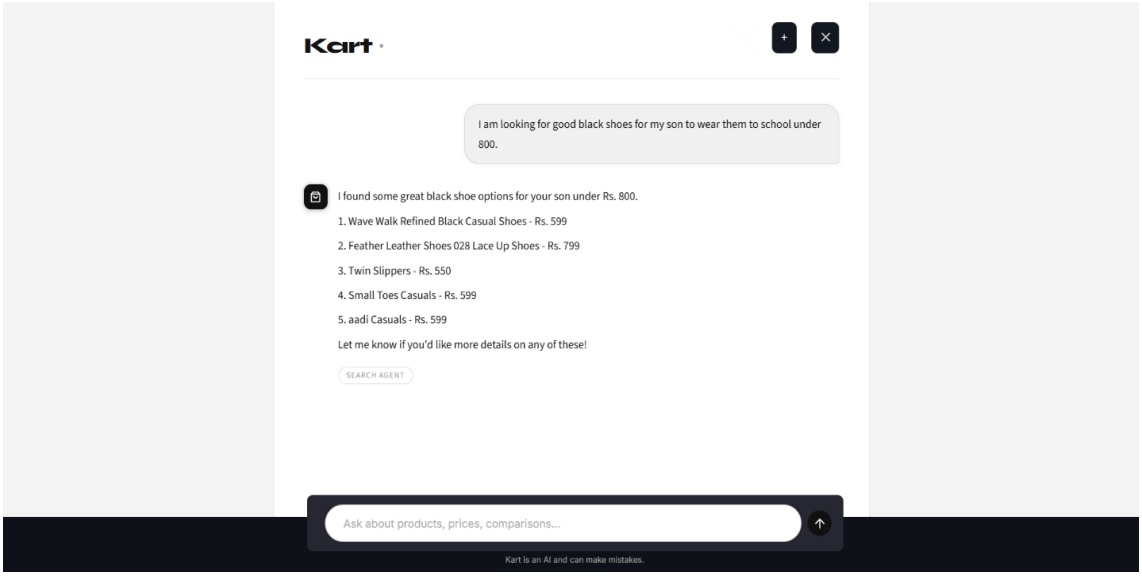

Figure 20: RAG Agent Scenario

Data Analyst Agent

The data analyst agent handles queries that require computation over the tabular product data, like calculating averages, counting products, ranking by price and comparing brands. The data analyst agent only relies on Pandas operations on the CSV.

The agent begins by parsing the user's query into a structured JSON specification using a dedicated LLM call, then it extracts parameters like brand, price, type and category. This part is critical as Pandas only understands exact instructions and the JSON specification provides that. The data analyst agent uses the LLM only to return a JSON with no additional text as shown in **Figure 21.**

```
_PARSE_TEMPLATE = """Extract query parameters from this e-commerce analytical query.

Available operations:
- "top_cheapest"   : return the N cheapest products (sort by discounted_price ASC)
- "top_expensive"  : return the N most expensive products (sort by discounted_price DESC)
- "filter"         : filter by category/brand/price, return results sorted cheapest first
- "average_price"  : compute the average discounted price for a category or brand
- "count"          : count how many products match the criteria
- "brand_count"    : show how many products each brand has (within a category if given)

OPERATION SELECTION RULES (follow strictly):
- Any query containing "brand", "brands", "which brand", "best brand", "top brand", "popular brand" → use "brand_count"
- Any query asking for cheapest/lowest/affordable → use "top_cheapest"
- Any query asking for most expensive/premium/luxury → use "top_expensive"
- Any query asking for average/mean price → use "average_price"
- Any query asking how many/count → use "count"
- Everything else → use "filter"

Known categories (use exact spelling if mentioned, else null):
Clothing, Jewellery, Footwear, Mobiles & Accessories, Automotive,
Home Decor & Festive Needs, Beauty and Personal Care, Home Furnishing,
Kitchen & Dining, Computers, Watches, Baby Care, Tools & Hardware,
Toys & School Supplies, Pens & Stationery, Bags Wallets & Belts,
Furniture, Sports & Fitness, Cameras & Accessories, Gaming, Electronics

Return ONLY this JSON:
```

Figure 21: Analyst Parsing Query Code

After the JSON specification is extracted, the next processing is done by Pandas without any involvement from the LLM, like if the user requests the cheapest product of a certain

criteria, the agent runs the top_cheapest function as shown in **Figure 22.**

```
if operation == "top_cheapest":
    result_df = (
        filtered.dropna(subset=["discounted_price"])
        .sort_values("discounted_price", ascending=True)
        .head(top_n)
    )
    return {
        "summary": f"Top {len(result_df)} cheapest {_label(spec)}:",
        "products": _to_product_list(result_df),
    }
```

Figure 22: Top Cheapest Function Code

This process ensures that everything returned to the user is only retrieved by Pandas operations from the CSV. **Figure 23** shows the data analyst agent responding to the query "what is the average price of clothing?" and "what are the top clothing brands".

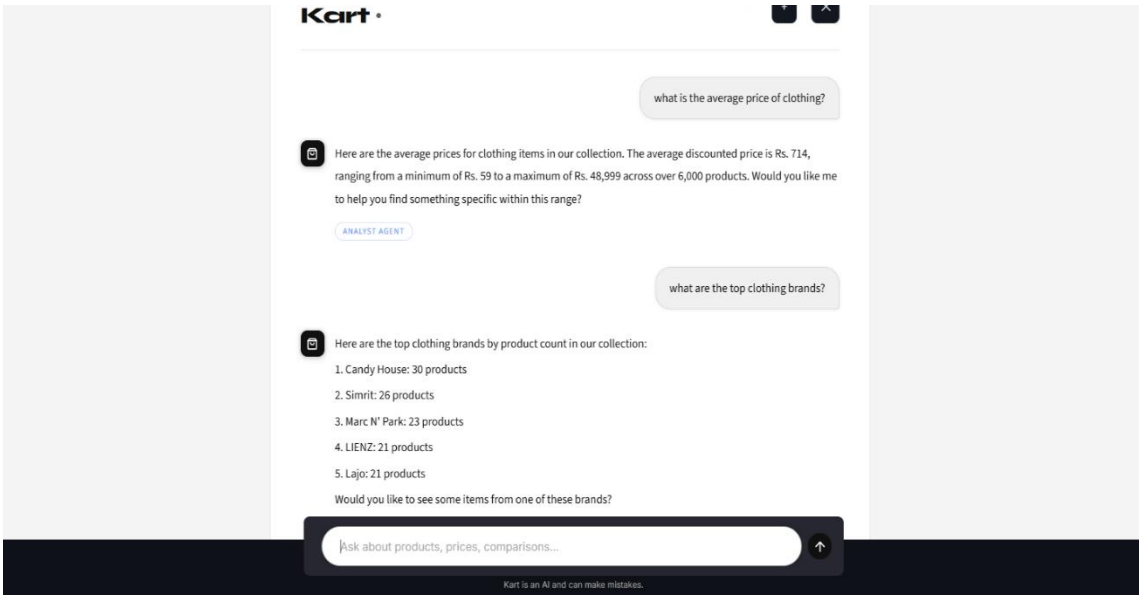

Figure 23: Analyst Agent Scenario

After the implementation and demonstration of our framework, the next subchapter is the quantitative evaluation of this framework against our functional and non-functional requirements.

9.5 Framework Evaluation

This subchapter evaluates our framework against its functional and non-functional requirements using three experiments based on the order of the components, the first measures the routing tool accuracy, the second measures the end-to-end latency at each stage of our framework and the third evaluates hallucinations. All experiments were done on the same hardware and using the same tools discussed earlier in our tools used chapter.

Experiment 1

The first experiment tests how accurately the router chooses the right agent for the task as wrong routings can cause worse results for the end user.

160 labelled queries covering the three cases search, analytical and chitchat were used, these queries included comparisons, product lookups, price follow ups and ambiguous phrasing. Each query was then compared to the correct answer labelled beforehand. **Table 2** shows the results

Class	Precision	Recall	Support
search	1.000	1.000	86
analytical	0.979	1.000	47
chitchat	1.000	0.963	27
Weighted average	0.994	0.994	160

Table 2: Routing Accuracy Test Results

Precision is how often the router was right per class, recall is how many did the router get correct out of the queries presented per class and support is how many queries belong to that class. The router achieved an overall accuracy of 99.4%, with 100% precision on search and chitchat. However, "what categories do you cover" query was misclassified which was supposed to be chitchat but was routed to the analytical agent. These results showed that this LLM model can route with high accuracy but can sometimes struggle with unclear queries. **Figure 24** shows the confusion matrix of the previous results.

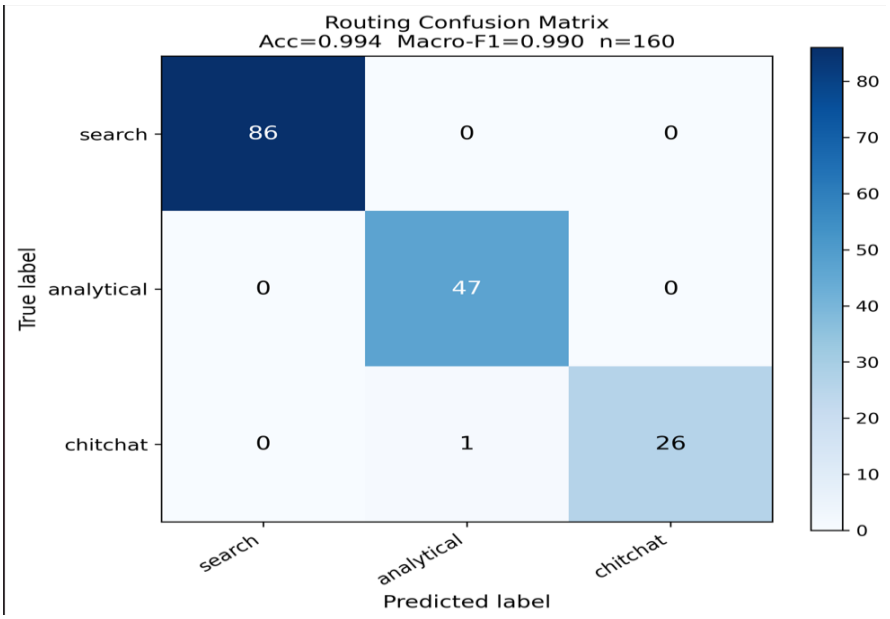

Figure 24:Routing Accuracy Confusion Matrix

Experiment 2

As this framework is designed to run locally, response time is a central concern for the user experience, to characterize where time is spent, each query had four measurement points, t_route for router call, t_retrieve for either RAG search or analyst search, t_generate for the final LLM formatting and t_total for the full time.

A subset of 20 queries was drawn from the queries used for the router experiment with 10 being search and 10 being analytical. The first two queries were discarded as warmups for cache effects and a timeout of 120s was set with one query exceeding that timeout, leaving a total of 17 queries. **Table 3** shows the results.

Agent	Stage	n	Mean (s)	Median (s)	Std (s)
search	t_route	8	19.12	18.89	0.63
search	t_retrieve	8	1.42	0.62	1.58
search	t_generate	8	53.64	53.66	3.58
search	t_total	8	74.18	74.25	4.36
analytical	t_route	9	19.78	20.02	0.62
analytical	t_retrieve	9	31.00	31.18	1.02
analytical	t_generate	9	30.75	28.44	6.23
analytical	t_total	9	81.53	80.33	6.27

Table 3: Latency Test Results

The Agent covers which agent handled the query, stage covers which part of was being timed, n covers the number of queries measured for that row, mean is the average for measurements, median is the middle value for the times when sorted, std is the standard deviation which is how spread out the measurements are.

The routing stage was consistent across both agents in the 19-20s, the retrieval was negligible in the search agent with a 0.62s median but much higher in the analyst agent with a 31.18 median as it invokes a second LLM call to structure the user's query into a Pandas query.

The total median time was similar in both agents with 74.25 at the search agent and 80.33 at the analyst agent, both are high for an interactive interface and would directly affect the customer experience negatively, this could be mitigated by the use of a smaller model or better hardware, this experiment outlines our hardware limitations as all of this is running on a CPU only computer with no GPU acceleration. All the steps with high latency include LLM calls, which are improved with better hardware, another limitation

is our small latency sample also due to the hardware limitations.

Experiment 3

The third experiment evaluates against our main non-functional requirement to mitigate hallucinations, which means to not invent products or retrieve products out of the Flipkart dataset in this scenario. This experiment tests our framework against the raw LLM model Qwen3.5:4b without any RAG or agents.

A response was classified into one of four categories. A response is grounded and present in Flipkart dataset, fabricated if not present in dataset, wrong but real if the named products exist in the dataset but do not satisfy the user's query and honest refusal if the system states it could not find the product. An evaluation set of 40 queries was constructed in three categories, 15 queries asking for products present in the dataset, 15 queries asking for real products that are not present in the dataset and 10 adversarial like instruction overrides and roleplay bypasses.

System	In-catalogue	Out-of-catalogue	Adversarial	Overall fabrication
Solution (full framework)	0%	0%	0%	0.0%
Baseline (raw qwen3.5:4b)	0%	40%	40%	25.0%

Table 4: Hallucination Test Results

Across the 40 queries our framework did not fabricate products under any conditions, while the baseline fabricated 10 out of the 40 cases, with all of them being out of catalogue or adversarial. This validates our non-functional requirement with 0% fabrication shown by our solution against the baseline of 25% showing a significant difference.

The first experiment shows that our framework is accurate and meets its routing functional requirements of correct agent selection and multi agent requirement. Experiment 2 validates the local deployment and interactive requirements but weakness in our latency experiment due to hardware limitations. Experiment three validates the main requirement of mitigating hallucination, in the next subchapter expert interviews will be conducted with experts relevant to our framework to evaluate our project from a different perspective.

9.6 Expert Interviews

According to our solution approach, in this subchapter we evaluate our demonstrated framework using the feedback of field experts to identify any issues in our design with the use of structured expert interviews.

Interview Approach

Our interview approach follows a semi-structured written style. Each participant is provided with the results and demonstration chapter of our solution, that documents the tools used, dataset, code snippets and user interaction scenarios. All experts are provided with the same documentation and standardized questions.

To ensure that the evaluation includes both the technical and the commercial aspect of the framework, the interview is informed by the Technology-Organization-Environment (TOE) framework, which argues that the characteristics of the technology artifact are shaped by the technology, the organizational context and the environment (Baker, 2012; p.232). The open-ended question design allows each expert to evaluate in the area which their professional judgement carries the most weight while still maintaining the same shared questions for comparability.

Experts were chosen by their relevance to our project, either field experts or academic experts, such as business informatics professors and teaching assistants, AI researchers and software developers, moreover experts are independent from the supervision of this thesis to avoid bias and maintain integrity. The panel of experts is small, but each expert delivers a different perspective.

The interview questions were chosen to be open-ended and concise, to respect the experts' time and receive different answers rather than yes or no questions. The interview consists of four questions that are as follows:

1. Based on the attached chapter, how would you summarize your overall impression of this system and its potential within the e-commerce space?

2. What are your thoughts on the multi-agent design?

3. What are the most promising strengths and the most notable limitations of this approach?

4. Are there any improvements you would recommend for future iterations of the system?

This interview was done over e-mail due to time constraints, each expert received an invitation message summarizing the research context and outlining the use of their responses, the demonstration chapter was sent as an attachment to the e-mail, after that, if the expert agreed responses were delivered also via e-mail.

Expert 1

The first expert is a former full stack software engineer and the co-founder of a tech startup company. Their answers went as follows:

Q1: Expert 1 said that the project is well structured with high potential for commercial solutions, suggesting it could help e-commerce platforms improve customer experience.

Q2: The expert liked the multi-agent design but mentioned that it would be hard to scale as it is all running locally.

Q3: The customer facing property was mentioned as the main strength, but the main limitation is the dataset due to its small size.

Q4: The expert proposed saving user messages to improve conversational context and analyzing the most frequent questions for re-training to improve performance and integrating the system into a working e-commerce platform.

Expert 2

The second expert is a master's degree holder in AI and is currently working as an AI software engineer.

Q1: They described it as a competent and reproducible prototype built on an open-source stack and acknowledged the framework for its awareness of hallucination.

Q2: The router and agents were judged as well motivated and cleanly implemented, with credit given to the data analyst agent as the stronger element of the design as it produces a structured JSON specification and delegates it to Pandas. The search agent was credited for its grounding, but they commented that the agents are not autonomous.

Q3: The highlighted strengths were about the grounding of the dataset and the reproducibility of the analytical answers. The highlighted limitation was that the data cleaning was insufficiently justified as imputing prices with the categorical means biasing the analytical queries and imputing null brand names with generic which distorts brand count operation.

Q4: The expert recommended revisiting the data cleaning pipeline.

Expert 3

The third expert is a former Marketing director and currently is the head of business development at his respective organization, giving insight into the commercial aspect of the solution.

Q1: They described it as a helpful tool that can help the customer find what he needs while increasing efficiency and matching the technology sales advancements in e-

commerce.

Q2: They praised the multi-agent design but commented that the analytical agent is the stronger part of the design as it can increase efficiency for choosing items based on certain price ranges.

Q3: The outlined strength was saving effort for customers while helping them expand their search by seeing different items than the ones they had in mind, increasing sales for the platform. The main limitation is that it must be linked to one catalogue making the product range smaller but could benefit loyal customers.

Q4: They expressed that for future work this framework could be connected to multiple e-commerce websites at the same time expanding the product catalogue for customers.

Expert 4

The fourth expert is an assistant lecturer of business informatics at the German University in Cairo, giving feedback both from the technical and commercial perspective.

Q1: Expert 4 described the framework as effective as it supports follow-up queries in a conversational way and has good privacy due to it being fully local.

Q2: They described the multi-agent design as a good choice as it ensures accurate responses by combining the tools of both agents.

Q3: They praised privacy as the strength of this framework but as for the limitations, they mentioned that it could be compared to similar systems as a benchmark.

Q4: They recommended that it is evaluated against similar systems and implementation in sensitive areas like healthcare and legal services.

9.7 Framework Refinement

Following our solution approach, the next part after the evaluation is the refinement of the framework using the feedback given by the experts. This subchapter documents the refinement of the framework by revisiting the data cleaning pipeline in response to the concerns raised by expert 2 as this concern directly impacts the retrieval quality which is essential for a recommendation system.

Expert 2's evaluation identified two issues with the original cleaning pipeline. Firstly, is the imputation of retail_price and discounted_price values using the mean which could introduce bias due to a right skewed price distribution as expensive items can pull the mean outward. Secondly, imputing null brand values with generic was judged to distort the brand count operations, both issues were fixed in this refinement.

The cleaning pipeline was fully rebuilt to address all the issues above and for additional improvements. Firstly, the prices were replaced by the median of their respective

categories as addressed by expert 2's concerns. The null values in brand names were kept as null instead of adding generic to not distort brand count operations. Moreover, a new Boolean column named price_imputed was added to enable the agents to differentiate between imputed and non-imputed prices.

This leaves the new cleaned dataset with 19,998 records as 2 records were removed for being empty and 78 price values were imputed with the median of their category and 10 columns being product_name, primary_category, retail_price, discounted_price, price_imputed, description, brand, specifications_text, product_rating and overall_rating.

This refined dataset was then re-embedded using the same embedding model and is compatible with both the RAG search agent and the data analyst agent closing the gap identified by expert 2.

After building our solution and evaluating it both qualitatively and quantitatively, the next chapter summarizes the findings of this thesis and outlines both the limitations and future work.

10 Conclusion

The objective of this thesis was to develop a recommendation system that uses the personalization and reasoning capabilities of LLMs while making sure that all generated outputs map to existing inventory items. To reach this objective we researched the most popular design science research methodologies and created our own solution approach that follows a mix of the discussed methodologies, which structures the research process into 3 phases starting with the problem identification, design and then implementation and evaluation. The problem identification phase was done through a literature review of state of the art recommendation systems like the use of LLMs, GNN and CBF, from this review we identified the problem as hallucinations, specifically when the recommendation system generates items that are non-existent or do not map to existing inventory items, which led to our objective mentioned above.

After the problem identification phase, we went into the design phase where we analyzed partial solutions of our gap to extract five functional requirements being natural language understanding, grounded product search, analytical query handling, query routing and a conversational ability, we then extracted three non-functional requirements hallucination mitigation, local deployability and modularity, these requirements informed our structural and behavioral models for our proposed solution, our solution is a multi-agent AI recommendation system made up of an LLM router, RAG search agent, data analyst agent and a product knowledge base. The structural model defined each component and its responsibilities, while the behavioral model described the lifecycle of the query across all the six stages and as the structure separated LLM reasoning from retrieval and all outputs were verified against the knowledge base, this architecture prevented hallucinations. We then went into the evaluation phase where we first implemented our framework on local hardware using open-source tools and a Flipkart dataset made of 19,998 products after being prepared.

We then demonstrated our framework across different user scenarios and showed code snippets of relevant functions, after that, we evaluated our framework using three experiments, the first experiment measured router accuracy using 160 labelled queries where our framework achieved 99.4% accuracy, the second experiment measured end-to-end latency across all stages which achieved a median response time of 74.25 seconds for the search agent and 80.33 seconds for the analyst agent showing our hardware limitations, the third experiment measured hallucinations against a baseline of the same model without any agents where our framework did not fabricate any responses across 40 queries. After that, we evaluated our framework using expert interviews where we got feedback from experts across different industries which helped us refine our framework based on their feedback where we revisited the dataset cleaning pipeline due to issues outlined in the interviews.

In conclusion, the evaluation results and interviews show that the objective was achieved as the framework meets the functional and non-functional requirements and can mitigate hallucinations as mentioned in our objective.

10.1 Limitations

The limitations of this thesis are made up of two categories, first are limitations imposed by the environment and second are limitations by the conceptual scope of the framework.

This framework was developed and tested on a laptop without a dedicated GPU which is the main hardware limitation that caused increased latency while using local LLM models as shown in our second experiment, moreover this led us to use Qwen3.5:4b which is a good model but we used the 4b model due to having only 16GB of RAM, while there are 9b and 27b and there is a newer Qwen3.6 model but its smallest version as of now is 27b, all of the mentioned models would provide better overall performance but could not be used as they would not fit on 16GB of RAM, moreover, we had to disable thinking on our model as it increases latency which would improve performance, larger embedding models would also give better performance but could not be used with our hardware.

There are also conceptual limitations, mainly being the dataset as it has only 19,998 products from a single market and platform and is written only in the English language, which decreases the generalizability of the framework and limiting the scope. Moreover, the framework does not currently support memory across different chats, which prevents long term personalization. In addition, the framework still only exists as a prototype and is not linked with a live e-commerce platform, so performance under real conditions like inventory changes has not yet been tested, also scalability could be an issue as it is running locally which would require expensive and large infrastructure to scale this framework to larger projects. In addition, the dataset size used for the evaluations of routing, latency and hallucinations was small. Finally, a larger expert panel with a Delphi style process would give stronger evidence for the framework and more diverse feedback.

10.2 Future Work

Multiple directions for future work emerge from the expert interviews and the limitations discussed in the previous subchapter the main technical direction for future work is to use this framework on better hardware with a dedicated GPU and more RAM as this would improve latency and allow the use of larger models with more parameters. Moreover, future work should use larger and more diverse datasets and incorporate multiple product catalogues from different platforms in one knowledge base as informed by expert 3 which would allow comparisons across different platforms. In addition, this framework could be integrated into live e-commerce platforms as informed by expert 1, which would expose the framework to real inventory changes and users. Another important direction would be storing user chat history to improve personalization and help the model learn frequently asked questions.

Finally, this framework could be expanded into more than just e-commerce, especially in sectors where privacy is very important like legal and healthcare sectors as mentioned by expert 4.

Bibliography

Akter, S., & Wamba, S. F. (2016). Big data analytics in E-commerce: a systematic review and agenda for future research. Electronic markets, 26(2), 173-194

Aljunid, M. F., & Dh, M. (2020). An efficient deep learning approach for collaborative filtering recommender system. Procedia Computer Science, 171, 829-836

Baker, J. (2012). The Technology–Organization–Environment Framework. In Y. K. Dwivedi, M. R. Wade, & S. L. Schneberger (Eds.), *Information systems theory: Explaining and predicting our digital society, Vol. 1* (Integrated Series in Information Systems, Vol. 28, pp. 231–245). Springer

Benbya, H., Davenport, T. H., & Pachidi, S. (2020). Artificial intelligence in organizations: Current state and future opportunities. MIS Quarterly Executive, 19(4)

Bhangu, S., Provost, F., & Caduff, C. (2023). Introduction to qualitative research methods – Part I. *Perspectives in Clinical Research, 14*(1), 39–42

Cao, L., & Zhang, C. (2007). The evolution of KDD: Towards domain-driven data mining. *International Journal of Pattern Recognition and Artificial Intelligence*. University of Technology, Sydney

Cheatham, B., Javanmardian, K., & Samandari, H. (2019, April). Confronting the risks of artificial intelligence. McKinsey Quarterly, 1–9

Chen, H., Bei, Y., Shen, Q., Xu, Y., Zhou, S., Huang, W., Huang, F., Wang, S., & Huang, X. (2024). Macro graph neural networks for online billion-scale recommender systems. In Proceedings of the ACM Web Conference 2024 (pp. 3598–3608). Association for Computing Machinery

Deng, J., Chen, Q., Cheng, D., Li, J., & Liu, L. (2025). Logit space constrained fine-tuning for mitigating hallucinations in LLM-based recommender systems. In Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing (pp. 29299–29312). Association for Computational Linguistics

Duricic, T., Kowald, D., Lacic, E., & Lex, E. (2023). Beyond-accuracy: a review on diversity, serendipity, and fairness in recommender systems based on graph neural networks. Frontiers in big data, 6, 1251072

Ejjami, R. (2024). Enhancing user experience through recommendation systems: a case study in the ecommerce sector. International Journal for Multidisciplinary Research, 6(4), 6

Gao, C., Zheng, Y., Li, N., Li, Y., Qin, Y., Piao, J., Quan, Y., Chang, J., Jin, D., He, X., & Li, Y. (2023). A survey of graph neural networks for recommender systems: Challenges, methods, and directions. ACM Transactions on Recommender Systems, 1(1), Article 3

Garg, M. (2021). Overview of artificial intelligence. In No-Code AI: Concepts and Applications in Machine Learning (pp. 3–13). World Scientific

Ghanad, A. (2023). An overview of quantitative research methods. *International journal of multidisciplinary research and analysis*, *6*(08), 3794-3803

Gupta, S., Ranjan, R., & Singh, S. N. (2024). A comprehensive survey of retrieval-

augmented generation (RAG): Evolution, current landscape and future directions. Carnegie Mellon University

Haenlein, M., & Kaplan, A. (2019). A brief history of artificial intelligence: On the past, present, and future of artificial intelligence. California Management Review, 61(4), 5–14

Hevner, A. R., March, S. T., Park, J., & Ram, S. (2004). Design science in information systems research. *MIS Quarterly, 28*(1), 75–105

Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., & Chen, W. (2021). LoRA: Low-rank adaptation of large language models. arXiv

Huang, C., Zhang, Z., Mao, B., & Yao, X. (2023). An overview of artificial intelligence ethics. IEEE Transactions on Artificial Intelligence, 4(4), 799–816

Huang, J., & Wang, X. (2022). User Experience Evaluation of B2C E-Commerce Websites Based on Fuzzy Information. Wireless Communications and Mobile Computing, 2022(1), 6767960

Jain, V., Malviya, B., & Arya, S. (2021). An overview of electronic commerce (e-Commerce). Journal of Contemporary Issues in Business and Government, 27(3), 665–670

Karan, D., Chikwarti, D. K., & Ethan, A. (2024). Scalable Recommendation Engines Using Graph Neural Networks

Khan, S. N., & Sajjad, H. (2024). Personalization in e-commerce: Balancing consumer privacy and marketing effectiveness. *Research Corridor Journal of Engineering Science*, *1*(2), 135–149.

Kohavi, R., Mason, L., Parekh, R., & Zheng, Z. (2004). Lessons and challenges from mining retail e-commerce data. Machine Learning, 57(1), 83-113

Larsen, A. G., Skjuve, M. B., Følstad, A., & van As, N. (2025). LLM Hallucinations in Conversational AI for Customer Service: Framework and End-User Perceptions. *International Journal of Human–Computer Interaction*, 1–22

Li, Y., Fu, X., Verma, G., Buitelaar, P., & Liu, M. (2025). *Mitigating hallucination in large language models (LLMs): An application-oriented survey on RAG, reasoning, and agentic systems*. arXiv

Li, Y., Liu, K., Satapathy, R., Wang, S., & Cambria, E. (2023). Recent developments in recommender systems: A survey. arXiv

Madanchian, M. (2024). The impact of artificial intelligence marketing on e-commerce sales. *Systems*, *12*(10), 429

Massaroli, A., Martini, J. G., Lino, M. M., Spenassato, D., & Massaroli, R. (2017). The Delphi method as a methodological framework for research in nursing. *Texto Contexto Enfermagem, 26*(4), e1110017

Mittameedi, S. K., Dogra, V., & Sayal, A. (2025). Customer Experience in E-Commerce: A Systematic Review of Metrics, Models, and the Role of AI. IEEE Access

Naveed, H., Khan, A. U., Qiu, S., Saqib, M., Anwar, S., Usman, M., ... & Mian, A. (2025). A comprehensive overview of large language models. *ACM Transactions on Intelligent Systems and Technology*, *16*(5), 1-72

Nie, G., Zhi, R., Yan, X., Du, Y., Zhang, X., Chen, J., Zhou, M., Chen, H., Li, T., Cheng,

Z., Xu, S., & Hu, J. (2024). A hybrid multi-agent conversational recommender system with LLM and search engine in e-commerce. In *Proceedings of the 18th ACM Conference on Recommender Systems* (pp. 745–747). Association for Computing Machinery

Ning, L., Wang, S., Fan, W., Li, Q., Xu, X., Chen, H., & Huang, F. (2024). CheatAgent: Attacking LLM-empowered recommender systems via LLM agent. In Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (pp. 2284–2295). Association for Computing Machinery

Offermann, P., Levina, O., Schönherr, M., & Bub, U. (2009). Outline of a design science research process. *Proceedings of the 4th International Conference on Design Science Research in Information Systems and Technology (DESRIST'09)*, Malvern, PA, USA. ACM

Patil, P., Kadam, S. U., Aruna, E. R., More, A., Balajee, R. M., & Rao, B. N. K. (2024). Recommendation system for e-commerce using collaborative filtering. Journal Européen des Systèmes Automatisés, 57(4), 1145–1153

Peffers, K., Tuunanen, T., Rothenberger, M. A., & Chatterjee, S. (2007). A design science research methodology for information systems research. *Journal of Management Information Systems, 24*(3), 45–78

Qu, H., Fan, W., Zhao, Z., & Li, Q. (2025). TokenRec: Learning to tokenize ID for LLM-based generative recommendations. arXiv

Reddy, V. M., & Nalla, L. N. (2024). Real-time Data Processing in E-commerce: Challenges and Solutions. International Journal of Advanced Engineering Technologies and Innovations, 3(1), 297-325

Roy, D., & Dutta, M. (2022). A systematic review and research perspective on recommender systems. Journal of Big Data, 9, Article 59

Schröer, C., Kruse, F., & Gómez, J. M. (2021). A systematic literature review on applying CRISP-DM process model. *Procedia computer science*, *181*, 526-534

Sharma, C., Kaur, A., Datta, P., & Gulzar, Y. (2025). Optimizing eCommerce Data: Effective Approaches for Data Collection, Cleansing, and Preprocessing. In Strategic Innovations of AI and ML for E-Commerce Data Security (pp. 1-30). IGI Global

Sheikh, H., Prins, C., & Schrijvers, E. (2023). Artificial intelligence: Definition and background. In Mission AI: Research for Policy (pp. 15–41). Springer

Singh, P. K., Pramanik, P. K. D., Dey, A. K., & Choudhury, P. (2021). Recommender systems: An overview, research trends, and future directions. International Journal of Business and Systems Research,15(1), 14–52

Szadeczky, T., & Bederna, Z. (2025). Risk, regulation, and governance: Evaluating artificial intelligence across diverse application scenarios. Security Journal, 38, 35

Thorat, P. B., Goudar, R. M., & Barve, S. (2015). Survey on collaborative filtering, content-based filtering and hybrid recommendation system. International Journal of Computer Applications, 110(4)

Trinh, T., Nguyen, V.-H., Nguyen, N., & Nguyen, D.-N. (2025). Product collaborative filtering based recommendation systems for large-scale E-commerce.

International Journal of Information Management Data Insights, 5, Article 100322

Wang, Q., Li, J., Wang, S., Xing, Q., Niu, R., Kong, H., ... & Zhang, C. (2024). Towards next-generation llm-based recommender systems: A survey and beyond. arXiv preprint arXiv:2410.19744

Wang, Y., Jiang, Z., Chen, Z., Yang, F., Zhou, Y., Cho, E., Fan, X., Lu, Y., Huang, X., & Yang, Y. (2024). RecMind: Large language model powered agent for recommendation. *Findings of the Association for Computational Linguistics: NAACL 2024*

Wang, Y., Wu, M., Li, X., Xie, L., & Chen, Z. (2024). A survey on graph neural networks for remaining useful life prediction: Methodologies, evaluation and future trends. arXiv

Widayanti, R., Chakim, M. H. R., Lukita, C., Rahardja, U., & Lutfiani, N. (2023). Improving recommender systems using hybrid techniques of collaborative filtering and content-based filtering. Journal of Applied Data Sciences, 4(3), 289-302

Wu, X. K., Chen, M., Li, W., Wang, R., Lu, L., Liu, J., ... & Wang, F. Y. (2025). Llm fine-tuning: Concepts, opportunities, and challenges. *Big Data and Cognitive Computing*, *9*(4), 87

Yin, J., Qiu, X., & Wang, Y. (2025). The Impact of AI-Personalized Recommendations on Clicking Intentions: Evidence from Chinese E-Commerce. *Journal of Theoretical and Applied Electronic Commerce Research*, *20*(1), 21

Zhang, W., Bei, Y., Yang, L., Zou, H. P., Zhou, P., Liu, A., ... & Yu, P. S. (2025). Cold-start recommendation towards the era of large language models (llms): A comprehensive survey and roadmap. *arXiv preprint arXiv:2501.01945*

Zhang, W., & Zhang, J. (2025). Hallucination Mitigation for Retrieval-Augmented Large Language Models: A Review. *Mathematics*, 13(5), 856

Zhao, P., Zhang, H., Yu, Q., Wang, Z., Geng, Y., Fu, F., ... & Cui, B. (2024). Retrieval-augmented generation for ai-generated content: A survey. arXiv preprint arXiv:2402.19473

Zhao, Z., Fan, W., Li, J., Liu, Y., Mei, X., Wang, Y., Wen, Z., Wang, F., Zhao, X., Tang, J., & Li, Q. (2024). Recommender systems in the era of large language models (LLMs). *IEEE Transactions on Knowledge and Data Engineering, 36*(11), 6889–6907

Statement of Autonomy

This is to confirm that this work was created autonomously by Hesham Amr, using only the declared source of information and tools.

Hesham Amr

Cairo, May 2026